AI Ladder Evolution: Historical Classification, Resource Economics, and Structural Shifts Across Five Eras

Bottom-Line Summary: Artificial intelligence usage evolved between October 2018 and October 2026 through five distinct historical ladders (L_1 through L_5), each defined by a 10-fold to 100-fold expansion in single-task compute and a structural reorganization of human-AI workflows[1]. **Downward Capability Sinking:** Every 12 to 18 months, complex engineering workflows from the middle rungs of an existing ladder—such as custom Python document search pipelines in 2023—get absorbed into the default consumer chat interface at Step 1 of the next ladder[2]. **Runtime Compute Scaling:** The primary driver of capability shifted in late 2024 from training larger base models before deployment to spending orders of magnitude more compute during task execution through step-by-step verification and autonomous tool loops[7]. **Bimodal Resource Split:** Across all eras, 70% to 85% of active human users remain concentrated on the lowest two complexity steps, whereas the top two action-taking steps consume up to 1,000 times more text chunks (tokens), electricity in watt-hours (Wh), and dollar expenditure per individual run[4].

Structural Index

1. Part I — Literature & Cost Survey
   1.1. Ladder Literature Review
   1.2. Step Use-Case Sources
   1.3. Resource Metrics
2. Part II — Applied Ladder-Building Method
   2.1. Two-Shift Era Thresholds & Boundary Decisions
   2.2. Bottom-Up Clustering & Top-Down Pruning
   2.3. No-Overlap Step Split Verification
3. Part III — Chronological Ladder Evolution
   3.1. Era 1 Ladder (L1: 2018-10 to 2020-05) — Task-Specific Representation & Fine-Tuning
   3.2. Era 2 Ladder (L2: 2020-06 to 2022-11) — Foundation Completion & In-Context Prompting
   3.3. Era 3 Ladder (L3: 2022-11 to 2023-11) — Conversational Chat, Custom RAG & Unconstrained Agent Loops
   3.4. Era 4 Ladder (L4: 2023-11 to 2024-09) — Multimodal Context, Native Tool-Calling & Scaffolded Harnesses
   3.5. Era 5 Ladder (L5: 2024-09 to 2026-10) — Test-Time Compute Reasoning, Protocol Standardization & Autonomous Agents
4. Part IV — Overall Patterns & Summary Visuals

4.1. Long-Term Structural Patterns
4.2. Cross-Era Comparison & Visual Summary

1. Part I — Literature & Cost Survey

Bottom-Line Summary: Published AI frameworks from leading AI companies (frontier labs), venture capital firms, and open-source engineering communities divide along two axes: abstract capability ceilings versus concrete software control flow[1]. **Real-World Evidence Sources:** Reconstructing how practitioners actually used AI requires cross-referencing conceptual frameworks against annual enterprise spending surveys, developer questionnaires, educational course syllabi, and open benchmark leaderboards[2]. **Dual-Scale Measurement:** Accurate resource accounting demands tracking five physical and economic metrics—text chunks processed by the model (tokens), math operations (floating-point operations / FLOPs), electricity in watt-hours (Wh), financial cost (USD), and human-vs-AI duration—at both the single-task execution level and the industry-wide aggregate level[4].

1.1. Ladder Literature Review

Bottom-Line Summary: Existing AI ladders fall into three distinct traditions: frontier lab capability roadmaps, enterprise maturity and autonomy scales, and developer control-flow hierarchies[1]. **Theory vs. Practice:** Lab roadmaps frequently blend current product usage with speculative future thresholds such as autonomous corporations, whereas practitioner frameworks classify real-world software architectures deployed in production[1]. **Control-Flow Consensus:** By late 2024, industry literature converged on a sharp architectural boundary separating fixed multi-step code paths (workflows) from model-directed dynamic execution loops (agents)[7].

Software Shift Evolution: Former Tesla AI Director and OpenAI researcher Andrej Karpathy established the foundational developer ordering by dividing computing history into three eras[25]. **Explicit Code (Software 1.0):** Traditional engineering relies on human programmers writing explicit rules in languages such as C++, Java, or Python to process data predictably[25]. **Learned Weights (Software 2.0):** Machine learning replaces human-written rules with optimization over curated datasets to tune millions or billions of numerical parameters (neural network weights) for specialized tasks like image recognition or translation[25]. **Natural Language (Software 3.0):** Large language models (LLMs) treat plain English instructions and structured text memory windows (context windows) as programmable operating systems, shifting developer effort from weight optimization to context orchestration[25].

OpenAI AGI Roadmap: In July 2024, OpenAI disclosed an internal five-tier classification scale designed to measure progress toward human-level general capability (artificial general intelligence / AGI) by tracking how much human direction a system removes from the loop[1]. **Five Rung Progression:** The hierarchy ascends from conversational systems that respond to user prompts (Level 1: Chatbots), to models capable of PhD-level problem solving without external tools (Level 2: Reasoners), to systems that execute multi-day tasks across external software (Level 3: Agents), to systems generating novel scientific discoveries (Level 4:

Innovators), and finally to systems managing entire corporate operations (Level 5: Organizations)[1]. **Real-World Calibration:** Empirical audits of enterprise deployments through 2026 demonstrate that industry usage sits firmly around Level 2.5, where step-by-step reasoning tasks are routine, Level 3 agent deployments succeed in verifiable domains like software engineering but face high cancellation rates in unstructured business operations, and Levels 4 and 5 remain completely theoretical[1].

DeepMind Matrix Classification: Researchers at Google DeepMind (Morris et al., published November 2023 and updated through 2025) rejected single-axis ladders in favor of a two-dimensional grid separating capability depth (performance) from task breadth (generality), paired with a distinct six-level scale for human-AI interaction (autonomy)[14]. **Performance Rungs:** DeepMind ranks performance against skilled human adults across six tiers: Level 0 (No AI), Level 1 (Emerging: equal to an unskilled human), Level 2 (Competent: 50th percentile of skilled adults), Level 3 (Expert: 90th percentile), Level 4 (Virtuoso / Exceptional: 99th percentile), and Level 5 (Superhuman: outperforming 100% of humans, exemplified in narrow tasks by protein-folding system AlphaFold)[14]. **Autonomy Decoupling:** Crucially, Morris et al. demonstrated that model capability unlocks—but does not dictate—the deployed autonomy level, which spans Level 0 (human does everything), Level 1 (AI as a Tool), Level 2 (AI as a Consultant invoked by humans), Level 3 (AI as a Collaborator), Level 4 (AI as an Expert driving interaction), and Level 5 (AI as an Agent operating with full independence)[14].

Anthropic Architectural Split: In December 2024, Anthropic engineers Erik Schluntz and Barry Zhang published *Building Effective Agents*, establishing the definitive engineering classification used by production teams[7]. **Workflows vs. Agents:** Anthropic grouped all multi-step systems under the umbrella of action-taking systems (agentic systems) but drew a strict boundary between predefined code orchestration (workflows) and model-driven process control (agents)[7]. **Five Workflow Patterns:** Under predefined workflows, Anthropic cataloged five composable structures: (1) the augmented LLM (base model plus document search, tools, and memory), (2) prompt chaining (sequential calls with programmatic gate checks), (3) routing (classifying inputs to specialized prompts or smaller models), (4) parallelization (splitting tasks via sectioning or running multiple suitability checks via voting), and (5) orchestrator-workers (a central model dynamically delegating sub-tasks to worker models)[7]. **Autonomous Loops:** True agents occupy the top tier, where the model independently plans, selects abstract tools such as terminal execution (bash) or file modification (edit), inspects environmental feedback, and loops until verification criteria pass[5].

Enterprise & Autonomy Scales: Venture capital firms, industry analysts, and framework creators constructed parallel ladders focused on commercial adoption and control transfer[2]. **SAE Coding Adaptation:** Engineering teams adapted the automotive industry's self-driving car levels (SAE L0–L5) to software engineering: L0 (manual coding), L1 (single-line ghost-text autocomplete), L2 (chat-assisted function drafting), L3 (conditional multi-file editing under "plan, approve, execute" human supervision), L4 (background autonomous pull-request resolution in sandboxed test environments), and L5 (unsupervised full-lifecycle software engineering)[20]. **LangChain Autonomy Tiers:** LangChain CEO Harrison Chase formalized a practitioner autonomy spectrum progressing from fixed-code pipelines, to single router calls,

to state-machine workflows (LangGraph), up to unconstrained Deep Agents with guardrails removed[15]. **Gartner & Menlo Maturity:** Enterprise frameworks such as Gartner's five-level AI Maturity Model (Foundational, Emerging, Operational, Scaled, Transformational) and Menlo Ventures' enterprise stack reports track organizational progression from isolated consumer chat experimentation to deeply integrated departmental and vertical production systems[2].

Framework Name & Author	Publication Date	Core Ordering Axis	Number of Tiers / Levels	Key Contribution to Historical Ladder Construction
Software 1.0 / 2.0 / 3.0 (Andrej Karpathy)[25]	2017-11 / 2024-06	Programming interface & abstraction layer	3 Eras (Explicit Code, Neural Weights, Natural Language)	Explains the macro shift from training task-specific weights (L_1) to prompting foundation models (L_2–L_5)[25].
Levels of AGI & Autonomy (DeepMind / Morris et al.)[14]	2023-11 / 2025-09	Capability depth $\times$ breadth vs. Human-AI interaction	6 Performance tiers $\times$ 2 Generality columns; 6 Autonomy levels	Separates raw model capability from deployed product autonomy (Tool $\rightarrow$ Consultant $\rightarrow$ Collaborator $\rightarrow$ Agent)[22].
5 Levels of AGI (OpenAI Internal Roadmap)[1]	2024-07	Removal of human direction & task horizon	5 Levels (Chatbots, Reasoners, Agents, Innovators, Organizations)	Captures the exact 2024 transition from Level 1 Chatbots to Level 2

				Reasoners (o1) and Level 3 Agents[1].
Workflows vs. Agents (Anthropic Engineering)[7]	2024-12	Control-flow ownership (Code-driven vs. Model-driven)	1 Base + 5 Workflow patterns + 1 Autonomous Agent tier	Provides rigorous engineering boundaries separating Step 2 predefined workflows from Step 4/5 agents[7].
Cognitive Architectures (LangChain / Harrison Chase)[15]	2024-01 / 2025-07	State-machine flexibility & guardrail removal	5 Tiers (Code $\rightarrow$ LLM Call $\rightarrow$ Chain $\rightarrow$ Router $\rightarrow$ State Machine Agent)	Maps how brittle 2023 agent loops evolved into cyclic graph harnesses (LangGraph) and Deep Agents[35].
SAE Autonomy for Coding (Industry / Academic Survey)[20]	2023-08 / 2025-03	Scope of codebase control & human verification loop	6 Levels (L0 Manual $\rightarrow$ L1 Autocomplete $\rightarrow$ L3 Plan/Approve $\rightarrow$ L5 Autonomous)	Explains the 3-way split of AI coding into autocomplete, multi-file IDE diffs, and CLI/PR agents[5].

1.2. Step Use-Case Sources

Bottom-Line Summary: Populating historical ladders with verified real-world workflows rather than vendor marketing requires synthesizing four empirical archives: enterprise financial disclosures, developer tool surveys, technical course curricula, and standardized benchmark logs[2]. **Enterprise Spend Shift:** Corporate generative AI expenditure surged 21.7-fold in two years—from $1.7 billion in 2023 to $11.5 billion in 2024 (restated excluding raw inference, or $13.8 billion total) and $37.0 billion in 2025—while flipping from 47% internal custom builds in

2024 to 76% purchased off-the-shelf software in 2025[2]. **Curriculum Mirroring:** Engineering courses on platforms like DeepLearning.AI and Stanford CS224N/CS324 directly reflect the dominant practitioner bottlenecks of each year, shifting from sequential word tagging in 2019, to prompt templates and document chunking in 2023, to reasoning models and protocol-based coding agents in 2025–2026[5].

Annual Enterprise Reports: Longitudinal datasets from Menlo Ventures (*The State of Generative AI in the Enterprise* 2023, 2024, and 2025 editions, plus the *2026 State of Consumer AI*), the St. Louis Federal Reserve, McKinsey, and MIT NANDA document exactly which workflows reached production[2]. **Application Layer Dominance:** By December 2025, user-facing software applications captured $19 billion (51.4%) of the $37 billion enterprise market, while foundation model APIs, training systems, and data orchestration infrastructure captured $18 billion (48.6%)[2]. **Departmental Spend Concentration:** Within the $7.3 billion departmental application tier in 2025, software coding captured $4.0 billion (55% of departmental spend), followed by IT operations (10%), marketing (9%), customer success (9%), design (7%), and human resources (5%), while industry-specific vertical AI reached $3.5 billion led by healthcare at $1.5 billion[2]. **Return & Failure Audits:** Simultaneously, MIT's audit of 300 enterprise deployments and MIT NANDA's 2025 study revealed that while purchased or partner-built tools reached production usage in 67% of cases, only 33% of custom in-house builds succeeded, and over 80% to 95% of poorly scoped enterprise pilots delivered zero measurable impact to corporate operating earnings[1].

Developer Surveys & Benchmarks: Practitioner adoption across coding and data engineering is anchored by the *Stack Overflow Developer Survey* (recording 84% of developers using or planning to use AI tools by 2025), the *State of AI Engineering* reports (showing 62% of production agent teams adopting or planning to adopt standard tool plugs [Model Context Protocol / MCP] in 2025), and St. Louis Federal Reserve workforce tracking (showing 37.4% of US workers aged 18–64 using generative AI on the job, boosting hourly productivity by 33% during active use)[12]. **Objective Evaluation Suites:** Technical capability transitions across ladders are verified through standardized agentic benchmarks—most notably SWE-bench Verified (500 human-validated real GitHub issues), SWE-bench Pro (Scale AI's 1,865-task multi-language public and 276-task private commercial code suites), OSWorld-Verified (graphical computer use), Terminal-Bench 2.1 (command-line execution), and MCP Atlas (multi-tool orchestration)[10].

Educational Curricula & Cookbooks: University syllabi and industry code cookbooks provide an exact timestamped record of practitioner skill requirements[5]. **University Course Evolution:** Stanford's natural language processing curriculum evolved from teaching word vectors and sequence-to-sequence attention in CS224N (2018–2020), to launching CS324 (*Large Language Models* by Percy Liang and Tatsu Hashimoto in Winter 2022) focused on foundation model scaling and in-context prompting, to specialized 2026 university workshops on autonomous software agents[46]. **Practitioner Short Courses:** Andrew Ng's DeepLearning.AI platform (serving over 7 million learners across 70+ short courses co-built with OpenAI, Anthropic, LangChain, and LlamaIndex) and Anthropic's Claude Academy trace the exact sequence of developer tool stacks: starting with *ChatGPT Prompt Engineering for Developers* (calling

gpt-3.5-turbo), advancing to *LangChain: Chat with Your Data* and *Building and Evaluating Advanced RAG*, and culminating in 2025–2026 courses on *Reasoning with o1*, *Multi AI Agent Systems with crewAI*, *Building with the Claude API & MCP*, and *Building Adaptive AI Agents* with Claude Code[5].

1.3. Resource Metrics

Bottom-Line Summary: Measuring the cost of climbing an AI ladder requires five complementary units—text chunks (tokens), compute operations (FLOPs), electrical energy (Wh), monetary expenditure (USD), and human-vs-AI duration—because no single metric captures both hardware physics and workplace economics[8]. **Runtime Compute Crossover:** While pre-training compute for frontier models grew 5-fold per year (0.7 orders of magnitude annually) from 10^{22} FLOPs in 2020 to $10^{26}+$ FLOPs in 2025, runtime inference scaling allows practitioners to multiply single-task compute by 100 to 1,000,000 times (2 to 6 orders of magnitude) on verifiable math and coding tasks[3]. **Three-Tier Calculation Rule:** Whenever direct per-step disclosures are absent, resource estimates combine published model parameter counts and API price tables with standardized physical equations linking active parameters to forward-pass FLOPs and datacenter power draw[4].

Unit Trade-Off Comparison: Each candidate measurement unit exhibits specific strengths and blind spots across historical eras[3]. **Text Chunks (Tokens):** Tokens (~0.75 English words per chunk) directly govern API billing and context memory limits after June 2020, jumping from 50 tokens for a single line of code autocomplete to $1,200,000+$ tokens for a multi-agent research run, but were not standard billing units for pre-2020 statistical classifiers[4]. **Compute Operations (FLOPs):** Floating-point operations provide an era-agnostic physical measure of processor work across both 2018 CPUs/GPUs and 2026 AI clusters, governed during deployment by the dense transformer forward-pass approximation[8]:

$$\mathrm{C}_{\mathrm{deployment}} \approx 2 \times N_{\mathrm{active}} \times T_{\mathrm{total}}$$

where N_{active} represents active model parameters per token and T_{total} represents total tokens processed (adjusting for key-value memory caching [KV-cache] and sparse mixture-of-experts [MoE] routing)[4]. **Electrical Energy (Wh):** Watt-hours translate compute into physical grid demand, anchored by empirical measurements such as Luccioni et al. (2022) recording $\sim 4\ \mathrm{Wh}$ per query on the 176-billion-parameter BLOOM model, and hardware efficiency gains where GPU FLOP/s per watt improved by 34% per year ($1.34\times$ /year) since 2023[3]. **Monetary Cost (USD):** Dollar figures capture real commercial friction—reflecting both a 49% annual increase in chip performance per dollar ($1.49\times$ /year) and a 50% to 90% discount on cached input tokens—and must be normalized across pay-per-token API tiers and 20–

200/month flat-rate consumer subscriptions[3]. **Human-vs-AI Duration:** Tracking human prompting and verification minutes against autonomous AI execution seconds reveals the crucial workplace shift from human-in-the-loop typing to background task delegation[1]. **Compute & Price Trajectory:** Historical data from Epoch AI and provider pricing tables demonstrate how rising base capability paired with plummeting marginal execution cost to unlock multi-step agent ladders[3]. **Training vs. Runtime Trade-Offs:** Frontier model training compute expanded from $3 \times 10^{23}\ \text{FLOPs}$ for GPT-3 in 2020, to $10^{24}\ \text{FLOPs}$ for PaLM and Chinchilla in 2022, $2 \times 10^{25}\ \text{FLOPs}$ for GPT-4 in 2023, $5 \times 10^{25}\ \text{FLOPs}$ for GPT-4o in 2024, $2.5 \times 10^{26}\ \text{FLOPs}$ for GPT-4.5 and Claude Sonnet 4 in 2025, and up to $10^{27}\ \text{FLOPs}$ for Claude Opus 5.5 and GPT-5.4 in 2026[3]. Simultaneously, Epoch AI established that practitioners can trade off pre-training compute for runtime compute (inference compute) across three distinct spans: increasing runtime compute by 1 to 2 orders of magnitude (OOM) saves $\sim 1\ \text{OOM}$ in training compute on general language tasks; increasing runtime compute by 2 to $3\ \text{OOM}$ saves $\sim 2\ \text{OOM}$ in training compute on sequential step-by-step reasoning tasks; and increasing runtime compute by 5 to $6\ \text{OOM}$ via parallel sampling and checking (probability that at least one of k generated candidates is correct / pass@k) saves 3 to $4\ \text{OOM}$ in training compute on cheaply verifiable tasks like coding and formal mathematics[8]. **Marginal Token Deflation:** Concurrently, the API price per million tokens plunged by more than two orders of magnitude across six years: from $\$60.00/1\text{M}$ on GPT-3 davinci (2020-06) and $\$20.00/1\text{M}$ on text-davinci-003 (2022-11), down 10-fold to $\$2.00/1\text{M}$ on gpt-3.5-turbo (2023-03, while flagship GPT-4 launched at a $\$45.00/1\text{M}$ blended rate), falling to $\$5.00/1\text{M}$ on GPT-4o (2024-05), $\$9.00/1\text{M}$ blended on Claude 3.5 Sonnet (2024-06, with prompt caching cutting cached input token costs by 90% to $\$0.30/1\text{M}$), $\$3.50/1\text{M}$ blended (1.50 input / $9.00 output) on `Gemini 3.5 Flash` (`2026-02`), and reaching $\$0.18$–$\$0.20/\text{1M}$ output tokens on specialized coding models like InclusionAI's Ling 3.0 Flash and Poolside's Laguna S 2.1 by September 2026[10].

Milestone Date	Model / System Benchmark	Frontier Training Compute (FLOPs)	Runtime Inference Trade-Off Span	API Price per 1M Tokens (Blended / Tier)

2020-06	OpenAI GPT-3 (davinci) [cite: 3, 4, 54]	$\sim 3.1\times$	Single forward pass (0 OOM span)	$\$60.00/1\text{M}$ (launch rate)
2022-11	InstructGPT (text-davinci-003) [cite: 3, 4, 54]	$\sim 3.5\times$	Few-shot in-context ($+0.5$ OOM)	$\$20.00/1\text{M}$ ($3\times$ cut)
2023-03	gpt-3.5-turbo / GPT-4 [cite: 3, 4, 54]	$\sim 2.0\times$ (GPT-4)	$+1$–2 OOM general language tasks[8]	$\$2.00/1\text{M}$ (3.5) / $\$45.00/1\text{M}$ (GPT-4)
2024-06	GPT-4o / Claude 3.5 Sonnet [cite: 18, 54, 55, 57]	1.0–$5.0\times$ [cite: 18]	$+2$–3 OOM sequential reasoning[8]	$\$5.00$–$\$9.00/1\text{M}$ ($\$0.30$ cached input)[55]
2025-05	GPT-4.5 / Claude Sonnet 4 [cite: 3, 6, 18]	$\sim 2.5\times$	$+3$–5 OOM extended thinking & tool loops[6]	$\$3.00$ input / $\$15.00$ output (Sonnet 4)[6]
2026-09	Claude Opus 5.5 / GPT-5.4 / Laguna S 2.1 [cite: 3, 10, 54]	$\sim 1.0\times$	$+5$–6 OOM verifiable coding (pass@k)[8]	$\$20.00$ output (Opus 5.5) / $\$0.20$ output (Laguna S 2.1)[10]

Two-Level Cost Baseline: Combining Epoch AI compute disclosures, Menlo Ventures enterprise spending totals, and empirical energy benchmarks establishes the macro baseline across eras[2]. **Training vs. Inference Split:** In 2024, OpenAI spent approximately $7.0 billion on cloud compute rented from Microsoft—allocating $5.0 billion (71%) to research, experimental runs, and final training (such as $386million$–400 million for the GPT-4.5 final training run and $80million$–83 million for GPT-4o, GPT-4o mini, Sora Turbo, and o1 post-training) and $1.8billion$–2.0 billion (29%–30%) to deployment inference compute[3]. **Lifetime Compute Crossover:** As formalized by Oxford and Epoch AI researchers ($C_{\text{total}} \approx ND + C_{\text{post-training}} + dNI$

), total deployment inference compute overtakes pre-training compute once total deployed tokens ($d \times I$) exceed training dataset tokens ($D \approx 10^{13}$), a threshold crossed globally in 2024–2025 as enterprise inference volume surged and reasoning models multiplied tokens generated per query[2].

2. Part II — Applied Ladder-Building Method

Bottom-Line Summary: Constructing an objective historical sequence of AI ladders requires enforcing strict era transition rules, pairing bottom-up workflow clustering with top-down theoretical pruning, and verifying every step split against a quantitative three-variable test[1]. **Five Locked Eras:** Applying the two-shift structural threshold across the 2018–2026 historical archive locks exactly five chronological Era Ladders (L_1 through L_5)[4]. **Speculative Step Pruning:** Theoretical rungs lacking real-world adoption data—specifically OpenAI's Level 4 ("Innovators") and Level 5 ("Organizations") and SAE Level 5 ("Fully Unsupervised Software Engineering")—are set aside from the historical ladders to preserve factual accuracy[1].

2.1. Two-Shift Era Thresholds & Boundary Decisions

Bottom-Line Summary: A new Era Ladder (L_t) begins only when a technical breakthrough or 10-fold price reduction triggers at least two simultaneous structural changes across the ladder steps[1]. **Avoiding Model Churn:** Minor incremental checkpoint releases (such as moving from GPT-3.5 to GPT-3.5-turbo-0613 or Claude 3.5 Sonnet to Claude 3.7) are absorbed within their existing era rather than spawning fragmented ladders[45]. **Five Structural Inflections:** Between October 2018 and October 2026, exactly four watershed moments satisfied the multi-shift threshold, dividing the eight-year history into five distinct ladders[4].
Threshold Application Decisions: Each era boundary reflects a verified simultaneous collapse of an old step and birth of a new upper tier[4]. **Cutoff 1 (2020-06 — Launch of OpenAI GPT-3 API):** Transitioned the industry from L_1 (training task-specific encoders like BERT) to L_2 (prompting foundation completion models), simultaneously collapsing feature-engineered statistical classifiers and introducing zero-shot/few-shot in-context learning plus inline code autocomplete (Codex)[4]. **Cutoff 2 (2022-11 — Release of ChatGPT & gpt-3.5-turbo):** Combined a 10-fold API price drop (from $20.00/1M tokens on text-davinci-003 to $2.00/1M tokens on gpt-3.5-turbo) with dialogue tuning, sinking raw zero-shot completion into a universal consumer web chat UI (Step 1 of L_3) while spawning custom vector-database document search (retrieval-augmented generation / RAG, Step 3) and unconstrained AutoGPT agent loops (Step 5)[4]. **Cutoff 3 (2023-11 — GPT-4 Turbo 128k & Claude 2.1 200k Context):** Sunk basic single-file RAG into default chat file-upload buttons (Step 1 of L_4), eliminated brittle unconstrained AutoGPT loops in favor of predefined tool-calling and state-machine harnesses (LangGraph, Step 5), and split coding into multi-file code editors (Cursor, Step 4)[4]. **Cutoff 4 (2024-09 to 2024-11 — OpenAI o1-preview, Claude 3.5 Sonnet v2 Computer Use & MCP):** Introduced training models to think step by step before answering (reinforcement learning with

verifiable rewards / RLVR test-time compute, Step 2 of L_5), standardized tool connections via the Model Context Protocol (MCP, Step 3), and elevated autonomous command-line coding (Claude Code, Step 4) and graphical screen-clicking agents (Step 5) into production reality[5].

2.2. Bottom-Up Clustering & Top-Down Pruning

Bottom-Line Summary: Each era's steps were synthesized by first grouping hundreds of documented workflows from developer surveys, course notebooks, and enterprise reports into empirical clusters, then cross-checking those clusters against published lab classifications[1].
Capturing Practitioner Reality: Bottom-up clustering preserved high-volume engineering steps that abstract AGI roadmaps overlooked—such as training lightweight adapter layers on open models (parameter-efficient fine-tuning / PEFT and low-rank adaptation / LoRA), long-context prompt caching, and standardized protocol integration (MCP)[4]. **Pruning Speculative Tiers:** Top-down cross-checking removed theoretical autonomy tiers that generated zero verified usage in enterprise audits[1].
Empirical Workflow Grouping: In Pass 1, concrete workflows taught in DeepLearning.AI courses, Stanford CS324, and Anthropic cookbooks were grouped by their technical setup and execution loop[5]. For example, in 2023 (L_3), developer activities clustered naturally into five distinct groups: (1) typing questions into the ChatGPT browser tab, (2) chaining prompt templates via Python scripts, (3) chunking PDFs into Pinecone or Chroma vector stores with LangChain, (4) renting a single A100 GPU to run 4-bit quantized low-rank adapter training (QLoRA) on Llama-2-7B, and (5) running experimental AutoGPT loops[4]. **Theoretical Cross-Check:** In Pass 2, comparing these clusters against OpenAI's 5 Levels of AGI and DeepMind's Autonomy Levels confirmed the bottom and middle tiers ("Chatbots" / "AI as a Tool" and "AI as a Consultant") while exposing that OpenAI's Levels 4 and 5 ("Innovators" and "Organizations") had no empirical counterparts[1]. Conversely, Anthropic's December 2024 separation of predefined "Workflows" from dynamic "Agents" matched the exact divergence observed in 2024–2026 enterprise deployments[2].

2.3. No-Overlap Step Split Verification

Bottom-Line Summary: Every adjacent pair of steps (k and $k+1$) inside each ladder passes the Two-of-Three Step Split Test, guaranteeing that the ordering has no overlap and no gaps (mutually exclusive and collectively exhaustive / MECE)[4]. **Three Split Variables:** Two workflows occupy separate steps only if moving from Step k to Step $k+1$ alters at least two of the following dimensions: (1) task and use-case complexity scope, (2) tool and autonomy setup, and (3) single-task resource cost scale ($10\times$ to $100\times$ jump in tokens, FLOPs, or runtime)[4]. **Consolidation of Equivalents:** Workflows sharing identical scope, setup, and cost scales—such as routing and prompt chaining within Anthropic's predefined workflows—are combined into a single step to prevent artificial fragmentation[4].

Era Ladder ID & Valid Window	Era Ladder Title	Triggering Breakthroughs (Entry)	Getting Outdated Drivers (Exit)	Number of Verified Steps
L_1 (2018-10 to 2020-05)	Task-Specific Representation & Fine-Tuning Ladder	Google BERT release (Oct 2018) & HuggingFace transformers library[4].	OpenAI GPT-3 API (May/June 2020) proving zero/few-shot prompting eliminates per-task weight training[4].	4 Steps (P1 Rules $\rightarrow$ P4 Fine-Tuned Encoders)[4]
L_2 (2020-06 to 2022-11)	Foundation Completion & In-Context Prompting Ladder	OpenAI GPT-3 API (davinci), GitHub Copilot (Codex) preview (June 2021)[4].	ChatGPT / gpt-3.5-turbo (Nov 2022) cutting token costs $10\times$ and shifting UI from completion to chat[4].	4 Steps (P1 Autocomplete $\rightarrow$ P4 Instruction Tuning)[4]
L_3 (2022-11 to 2023-11)	Conversational Chat, Custom RAG & Unconstrained Agent Loops	ChatGPT launch (Nov 2022), GPT-4 (Mar 2023), Llama-2 (Jul 2023), LangChain / Pinecone[4].	GPT-4 Turbo 128k & Claude 2.1 200k (Nov 2023) sinking basic RAG into chat UI; AutoGPT loop failures[4].	5 Steps (P1 Web Chat $\rightarrow$ P5 ReAct Agent Loops)[4]
L_4 (2023-11 to 2024-09)	Multimodal Context, Native Tool-Calling & Scaffolded Harnesses	128k–2M context (Gemini 1.5 Pro), GPT-4o, Claude 3.5 Sonnet (Jun 2024), Cursor IDE[4].	OpenAI o1-preview (Sep 2024), Claude 3.5 Sonnet v2 Computer Use (Oct 2024), and MCP (Nov	5 Steps (P1 Multimodal Chat $\rightarrow$ P5 Scaffolded Harnesses)[4]

			2024)[4].	
L_5 (2024-09 to 2026-10)	Test-Time Compute Reasoning, Protocol Standardization & Autonomous Agents	OpenAI o1/o3/GPT-5.4, Claude Code CLI, MCP Linux Foundation standard, Sonnet 4.6/Opus 5.5[5].	Active frontier shift as of October 2026; constrained by commercial private-repo generalization and MCP security[10].	5 Steps (P1 Artifact Chat $\rightarrow$ P5 Multi-Agent & GUI Use)[4]

3. Part III — Chronological Ladder Evolution

Bottom-Line Summary: Across the eight-year span from October 2018 to October 2026, the AI usage ladder reorganized five times as foundation model parameters scaled from 10^8 to $10^{12}+$, context windows expanded 4,000-fold from 512 tokens to 2,000,000 tokens, and single-task agentic compute surged by 11 orders of magnitude (10^6 FLOPs to 9×10^{16} FLOPs)[3]. **Training to Prompting to Acting:** Practitioners progressed from training separate neural network weights for every narrow classification task (L_1), to prompting a single frozen foundation model via text completion (L_2), to wiring external vector databases and early agent loops (L_3), to embedding multimodal long-context models inside code editors and state-machine workflows (L_4), and finally to delegating multi-hour engineering and research tasks to reasoning models and protocol-standardized command-line and desktop agents (L_5)[4].

3.1. Era 1 Ladder (L1: 2018-10 to 2020-05) — Task-Specific Representation & Fine-Tuning

Bottom-Line Summary: During Era 1 (2018-10 to 2020-05), applying AI to text required data science teams to collect labeled datasets and train or fine-tune a separate model for every individual business task[4]. **Transformer Transfer Breakthrough:** Google's October 2018 release of bidirectional transformer encoders (BERT) replaced complex recurrent neural networks at the top of the ladder with pre-trained encoders fine-tuned on single cloud GPUs[4]. **Zero-Prompting Era:** Because models lacked generative instruction-following capabilities, 80% of industry practitioners relied on deterministic pattern rules (regular expressions) and tabular statistical classifiers (scikit-learn), while only 5% of users operated at the top fine-tuning tier[4].

Valid Dates & Why Old Ladder Got Outdated

Era Window (2018-10 to 2020-05): Era 1 opened in October 2018 when Google researchers published BERT and HuggingFace released its open-source PyTorch library (pytorch-pretrained-bert, later renamed transformers), establishing pre-train-then-fine-tune as the gold standard for natural language processing (NLP)[4]. **Outdating Pre-2018 Architectures:** Prior to October 2018, NLP ladders required building custom sequential recurrent networks with long short-term memory cells (BiLSTM) from scratch on top of static word lookup tables (Word2Vec or GloVe), which failed to capture word meaning changes across sentence contexts[4]. **Why L1 Ended in May 2020:** Era 1 became outdated in May–June 2020 when OpenAI published the 175-billion-parameter next-word prediction model (GPT-3) and launched its commercial completion API, proving that a single massive model could solve translation, classification, and drafting via plain-text examples in the prompt (in-context learning) without updating a single neural network weight[4].

How Steps Changed

Structural Realignment: Compared to pre-2018 NLP workflows, L_1 retained deterministic rules at Step 1 and classical statistical classifiers at Step 2, compressed static word embeddings and recurrent networks into Step 3, and introduced **New Top Step 4** (Fine-Tuned Transformer Encoders) where transfer learning on 110M–340M parameter models (BERT-base and BERT-large) achieved state-of-the-art accuracy on classification and extraction benchmarks[4].

Step-by-Step Breakdown (Steps 1 to 4)

Step 1 — Deterministic Rules & Regular Expressions (L1-P1): Step Boundaries: Step 1 represents the lowest task complexity floor (single pattern or keyword extraction), requiring zero training data or statistical inference whereas Step 2 requires labeled training examples and feature matrices, and failing completely when phrasing varies outside explicit human-coded patterns (the Software 1.0 limit)[4]. **Concrete Tool Stack:** Practitioners relied on the Python standard library re, spaCy rule-based Matcher and PhraseMatcher, NLTK tokenizers, Elasticsearch keyword analyzers, and SQL LIKE queries running on standard CPUs[4]. **Use Cases & Context Switching:** Software engineers and data analysts extracted dates, invoice numbers, email addresses, and profanity filters from server logs and customer forms; a practitioner used Step 1 regex for structured fields (such as phone numbers) before switching to Step 2 or Step 4 for unstructured sentiment or customer intent[4]. **Step Resource Use:** A single task uses 0 neural tokens, $\sim 10^6 \text{ FLOPs}$ of CPU string matching, $\$0.000001 \text{ USD}$, 0.00001 Wh of electricity, and 15 minutes of human regex crafting versus 0.001 seconds of runtime, while capturing 50% of active NLP practitioners, 0% of neural token volume, and 10% of total era spend[4].

Step 2 — Feature-Engineered Statistical Classifiers (L1-P2): Step Boundaries: Step 2 moves

from rigid string rules to statistical document classification across 100–5,000 word documents by converting text into word-frequency counts (bag-of-words / TF-IDF), but lacks sequential word-order awareness and fails when negation or word order flips meaning, which requires Step 3 or Step 4[4]. **Concrete Tool Stack:** Data teams deployed scikit-learn (TfidfVectorizer, LogisticRegression, MultinomialNB, LinearSVC), XGBoost, LightGBM, and Pandas running in Jupyter Notebooks on local laptops or AWS EC2 CPU instances[4]. **Use Cases & Context Switching:** Enterprise data scientists built spam filters, support ticket routers, and churn predictors taught in foundational Coursera and Fast.ai machine learning modules; practitioners always ran Step 2 as a fast baseline before justifying the GPU cost of Step 3 or Step 4[4]. **Step Resource Use:** A single task inference uses 0 neural tokens, $\sim 10^8 \text{ FLOPs}$, $\$0.00001 \text{ USD}$, 0.0001 Wh, and 30 minutes of human feature engineering and labeling versus 0.05 seconds of execution, representing 30% of users, 0% of neural tokens, and 25% of era spend[4].

Step 3 — Static Embeddings & Recurrent Sequences (L1-P3): Step Boundaries: Step 3 upgrades from word-count tables to dense numerical word vectors (300-dimensional embeddings) passed through sequential recurrent layers (BiLSTM) that track word order for word-by-word entity tagging (named entity recognition / NER) and machine translation, but is limited by memory decay over long paragraphs and an inability to distinguish words with multiple meanings (like "bank") across contexts, which requires Step 4 bidirectional attention[4]. **Concrete Tool Stack:** Engineers combined pre-trained Word2Vec (Google), GloVe (Stanford), fastText (Facebook), and spaCy v2 pipelines with custom PyTorch or TensorFlow 1.x BiLSTM-CRF architectures trained on NVIDIA K80 or P100 GPUs[4]. **Use Cases & Context Switching:** NLP engineers highlighted person, organization, and medical drug names in clinical notes and e-commerce search queries (core assignments in 2018–2019 Stanford CS224N); teams switched from Step 2 to Step 3 when word sequence mattered but latency budgets prohibited heavy transformer layers[4]. **Step Resource Use:** A single task processes ~ 200 word or subword tokens, consuming $\sim 10^{10} \text{ FLOPs}$, $\$0.0001 \text{ USD}$, 0.002 Wh, and 20 minutes of human pipeline tuning versus 0.2 seconds of runtime, accounting for 15% of users, 20% of neural token volume, and 30% of era spend[4].

Step 4 — Fine-Tuned Transformer Encoders (L1-P4): Step Boundaries: Step 4 represents the Era 1 complexity ceiling—adapting all 110M–340M parameters of a pre-trained bidirectional transformer (BERT) plus a custom task classification head on 1,000–100,000 labeled examples—but is strictly capped at 512 subword tokens and incapable of open-ended text generation or zero-shot task switching without retraining[4]. **Concrete Tool Stack:** Production stacks centered on HuggingFace transformers (bert-base-uncased, roberta-large, distilbert-base-uncased), Google Cloud TPUs or AWS p3.2xlarge instances (NVIDIA V100 16GB GPUs), the PyTorch AdamW optimizer, and ONNX Runtime for quantized serving[3]. **Use Cases & Context Switching:** Search engine teams (such as Google Search adopting BERT in late 2019),

legal contract reviewers, and biomedical researchers used Step 4 for extractive question answering (SQuAD) and semantic similarity, reserving it for high-value production endpoints after Step 1–3 baselines plateaued[4]. **Step Resource Use:** A single inference task processes 256 subword tokens (capped at 512), requiring $\sim 1.7 \times 10^{11}\ \text{FLOPs}$, $\$0.001\ \text{USD}$ (including amortized GPU fine-tuning), $0.05\ \text{Wh}$, and $45\ \text{minutes}$ of dataset curation and training monitoring versus $0.5\ \text{seconds}$ of inference, serving 5% of practitioners while driving 80% of transformer token volume and 35% of era spend[4].

Full-Ladder Resource Totals (L1)

Aggregate Era Footprint (2018-10 to 2020-05): Across its 20-month window, L_1 encompassed approximately $1.5\ \text{million}$ active ML/NLP developers and data scientists globally, logging $\sim 450\ \text{million}$ human engineering hours[4]. Total era compute across training (such as BERT-large at $\sim 10^{20}\ \text{FLOPs}$ and T5-11B at $\sim 10^{22}\ \text{FLOPs}$) and deployment inference reached $\sim 5 \times 10^{23}\ \text{FLOPs}$, consuming $\sim 85,000\ \text{MWh}$ of datacenter electricity, $\sim 1.2\ \text{trillion}$ subword tokens, and $\sim \$450\ \text{million}$ in enterprise cloud GPU and NLP software expenditure[3].

3.2. Era 2 Ladder (L2: 2020-06 to 2022-11) — Foundation Completion & In-Context Prompting

Bottom-Line Summary: Era 2 (2020-06 to 2022-11) inverted the AI software stack by replacing per-task model training with natural language prompting of a single massive foundation model (GPT-3 and Codex)[4]. **Birth of Inline Coding:** The June 2021 launch of GitHub Copilot introduced real-time ghost-text code autocomplete inside code editors as the new Step 1 entry point, capturing 55% of active users[4]. **High-Cost Completion Barrier:** Because flagship davinci API calls cost $20.00 to $60.00 per million tokens and required carefully formatted text-completion prefixes rather than conversational dialogue, usage remained restricted to developers, copywriting startups, and enterprise ML teams[4].

Valid Dates & Why Old Ladder Got Outdated

Era Window (2020-06 to 2022-11): Era 2 began in June 2020 when OpenAI opened commercial API access to GPT-3 (davinci, curie, babbage, ada), followed in June 2021 by the GitHub Copilot technical preview powered by OpenAI's 12B-parameter Codex model, and in January 2022 by InstructGPT (text-davinci-001 through 003) trained with human preference feedback (reinforcement learning from human feedback / RLHF)[4]. **Why L1 Got Outdated:** Stanford's Center for Research on Foundation Models (CRFM) and course CS324 documented that a single 175B-parameter model prompted with three text examples (few-shot learning) matched or beat fine-tuned BERT models across dozens of tasks without requiring labeled

training sets or dedicated GPU servers[25]. **Why L2 Ended in November 2022:** On November 30, 2022, OpenAI released ChatGPT (gpt-3.5-turbo), which wrapped dialogue state management into a free consumer web interface and slashed API token pricing 10-fold from $20.00/1M tokens (text-davinci-003) to $2.00/1M tokens, instantly rendering raw text-completion prompting and standalone copywriting wrappers outdated[4].

How Steps Changed

Cross-Era Step Transitions: Between L_1 and L_2, three major structural shifts occurred across the ladder[4]. **Combined Legacy Steps:** L_1 Steps 1, 2, and 3 (Regex, TF-IDF classifiers, and BiLSTMs) were displaced from active AI application roadmaps as a single GPT-3 completion call handled classification, entity extraction, and sentiment simultaneously into **Zero-Shot Prompt Completion (L2-P2)** and **Few-Shot In-Context Demonstration Chains (L2-P3)**, establishing natural language programming (Software 3.0) as the core practitioner workflow[4]. **New Bottom Entry Step: Inline Ghost-Text Code Autocomplete (L2-P1)** emerged at the bottom of the ladder via GitHub Copilot in VS Code, giving millions of software developers low-latency 50-token predictions without leaving their code editor[4]. **Evolved Top Step:** L_1 's BERT encoder fine-tuning evolved into **Instruction Fine-Tuning & Task-Specific Head Tuning (L2-P4)** using sequence-to-sequence models (T5, BART) and OpenAI's Fine-Tuning API on curie and davinci[4].

AI LADDER HISTORICAL PARADIGM SHIFT • 2018–2020 VS. 2020–2022

Foundation Models Replaced Per-Task Weight Training With 1,000x Higher Inference Compute

Era 1 (Software 2.0: Weights) Era 2 (Software 3.0: Prompting) Hover or tap any step to trace consolidation pathways

Single-Task Compute Scale: Log_{10}(FLOPs per Task) 10^6 (1 MFLOP) 10^{11} (100 GFLOP) ⚡ ~1,000x Leap 10^{12} (1 TFLOP) 10^{15} (1 PFLOP)

ERA 1 (2018-10 TO 2020-05) — Task-Specific Weight Training

Software 2.0

Step 4 (P4): Fine-Tuned Transformer Encoders (BERT) — 5% users
Hugging Face Transformers / BERT-Base / RoBERTa
1.7×10^{11} FLOPs
Task Cost: **$0.001** Human: **45 min** / AI: **0.5 s**

Step 3 (P3): Static Embeddings & Recurrent Sequences — 15% users
GloVe / Word2Vec / PyTorch BiLSTM
1.0×10^{10} FLOPs
Task Cost: **$0.0001** Human: **20 min** / AI: **0.2 s**

Step 2 (P2): Feature-Engineered Statistical Classifiers — 30% users
scikit-learn / TF-IDF / XGBoost / LightGBM
1.0×10^{8} FLOPs
Task Cost: **$0.00001** Human: **30 min** / AI: **0.05 s**

Step 1 (P1): Deterministic Rules & Regular Expressions — 50% users
Regex / Python re / SpaCy Rule-based Matching
1.0×10^{6} FLOPs
Task Cost: **$0.000001** Human: **15 min** / AI: **0.001 s**

ERA 2 (2020-06 TO 2022-11) — Foundation Model Prompting

Software 3.0

Step 4 (P4): Instruction Fine-Tuning & Head Tuning — 5% users
OpenAI Fine-Tuning API / PromptSource / FLAN
2.0×10^{15} FLOPs
Task Cost: **$0.25** Human: **60 min** / AI: **20.0 s**

Step 3 (P3): Few-Shot In-Context Demonstration Chains — 15% users
GPT-3 175B (davinci) / early LangChain 0.0.1
4.2×10^{14} FLOPs
Task Cost: **$0.072** Human: **10 min** / AI: **5.0 s**

Step 2 (P2): Zero-Shot Prompt Completion & Transformation — 25% users
OpenAI text-davinci-002 / Playground Web UI
1.05×10^{14} FLOPs
Task Cost: **$0.018** Human: **3 min** / AI: **2.0 s**

Step 1 (P1): Inline Ghost-Text Code Autocomplete — 55% users
GitHub Copilot / OpenAI Codex 12B / VS Code
1.2×10^{12} FLOPs
Task Cost: **$0.0005** Human: **0.2 min** / AI: **0.3 s**

CROSS-ERA TRANSITION **Structural Consolidation: Offline Training to Runtime Prompts**

Human Effort vs. AI Runtime	Single-Task Cost & Energy	Primary Mechanism Shift
Human minutes reduced; GPU inference seconds increased	**Era 1 under $0.001 → Era 2 up to $0.25 ($10^3$–$10^4$x cost/query)**	**Zero-shot attention in 175B weights replaces offline PyTorch pipelines**

Key Architecture Takeaway: In Era 1, 80% of real-world use was zero-token (rules, TF-IDF, XGBoost) and deep learning required offline weight fine-tuning. The arrival of GPT-3 in June 2020 collapsed deterministic rules and recurrent neural nets into frozen prompt completion calls—trading lightweight offline code for massive runtime inference compute.

The June 2020 arrival of GPT-3 and Codex shifted computational work from offline model training to runtime prompt execution. Whereas 80% of Era 1 practitioners relied on zero-token rules and statistical classifiers requiring under 100 million FLOPs per run, Era 2 moved 95% of users onto foundation model completion calls consuming between 1.2 trillion and 420 trillion FLOPs per task.

Data sources: Hugging Face (Karpathy Software 3.0), Epoch AI, PricePerToken, Stanford CS324

Step-by-Step Breakdown (Steps 1 to 4)

Step 1 — Inline Ghost-Text Code Autocomplete (L2-P1): Step Boundaries: Step 1 is the lowest complexity step in L_2, predicting 1 to 3 lines of code (~50 tokens) automatically from cursor context without explicit prompt writing, whereas Step 2 requires formulating natural language instructions, and Step 1 fails on whole-function architecture or cross-file logic due to narrow local context (~1,000–2,000 tokens) and 300-millisecond latency caps[4]. **Concrete Tool Stack:** Developers installed the GitHub Copilot extension inside Visual Studio Code, JetBrains IDEs, and Neovim, powered by OpenAI Codex (code-cushman-001 12B and code-davinci-002), alongside TabNine[4]. **Use Cases & Context Switching:** Software engineers auto-completed boilerplate loops, unit test assertions, and SQL queries; a developer used Step 1 continuously while typing code, then switched to the browser Playground at Step 2 or Step 3 when stuck on an unfamiliar algorithm or regex pattern[4]. **Step Resource Use:** A single task processes ~ 50 tokens, $\sim 1.2 \times 10^{12}\ \text{FLOPs}$ ($2 \times 12\text{B} \times 50$), $\$0.0005\ \text{USD}$ (prorated across a $10/month subscription), $0.02\ \text{Wh}$, and $0.2\ \text{minutes}$ (12 seconds) of human review versus $0.3\ \text{seconds}$ of AI generation, capturing 55% of active era users, 25% of tokens, and 20% of era spend[4].

Step 2 — Zero-Shot Prompt Completion & Text Transformation (L2-P2): Step Boundaries: Step 2 moves from passive code autocomplete to explicit natural language instructions for single-paragraph drafting, summarization, or classification (~300 tokens), differing from Step 1 by requiring deliberate prompt design and from Step 3 by omitting in-context input-output examples, while breaking down when tasks require strict output schemas (like nested JSON) or multi-step logic[4]. **Concrete Tool Stack:** Practitioners used the OpenAI Playground web interface, the OpenAI Completion API (text-davinci-001, text-davinci-002, text-davinci-003, text-curie-001), and early commercial SaaS wrappers including Jasper.ai, Copy.ai, and AI Dungeon[4]. **Use Cases & Context Switching:** Marketing copywriters generated ad variations, product managers summarized meeting notes, and early adopters explored InstructGPT; users started with a zero-shot prompt at Step 2 and escalated to Step 3 few-shot examples whenever davinci broke the output format[4]. **Step Resource Use:** A single task uses ~ 300 tokens, $\sim 1.05 \times 10^{14}\ \text{FLOPs}$ ($2 \times 175\text{B} \times 300$), $\$0.018\ \text{USD}$ (at $60/1M tokens on davinci or $0.006 at $20/1M on davinci-003), $1.5\ \text{Wh}$ [19], and $3\ \text{minutes}$ of human prompt tweaking versus $2.0\ \text{seconds}$ of AI runtime, representing 25% of users, 30% of tokens, and 35% of era spend[4].

Step 3 — Few-Shot In-Context Demonstration Chains (L2-P3): Step Boundaries: Step 3 supplies 3 to 10 curated input-output pairs (~1,200 tokens) plus explicit step-by-step reasoning cues ("Let's think step by step", Kojima and Wei et al. 2022) inside the 2,048-to-4,096 token context window, overcoming Step 2 formatting and logic failures without altering weights, but

hitting a hard wall at the 4,096-token context ceiling and incurring $4\times$ higher per-call API bills[4]. **Concrete Tool Stack:** Production teams called the OpenAI text-davinci-002 and text-davinci-003 API with custom stop sequences (stop=["\n\n", "Input:"]), Cohere Generate API, AI21 Labs Jurassic-1, and early Python prompt-template scripts taught in Stanford CS324 (Winter 2022)[4]. **Use Cases & Context Switching:** AI engineers extracted structured JSON records from unstructured medical or financial text, translated natural language into complex SQL queries, and solved grade-school math word problems (GSM8K); engineers used Step 3 for production API pipelines where Step 2 zero-shot accuracy was below 70%, escalating to Step 4 fine-tuning only when few-shot examples exhausted the 4k context window or cost too much per call[4]. **Step Resource Use:** A single task processes $\sim 1,200$ tokens, $\sim 4.2 \times 10^{14}\ \text{FLOPs}$, $\$0.072\ \text{USD}$, $6.0\ \text{Wh}$, and $10\ \text{minutes}$ of human example curation versus $5.0\ \text{seconds}$ of runtime, accounting for 15% of users, 30% of tokens, and 30% of era spend[4].

Step 4 — Instruction Fine-Tuning & Task-Specific Head Tuning (L2-P4): Step Boundaries: Step 4 is the highest complexity step in L_2, updating the weights of an open-source encoder-decoder (T5, BART) or running OpenAI's supervised Fine-Tuning API on thousands of JSONL prompt-completion pairs to eliminate the repeated token cost of Step 3 few-shot examples during inference and lock in domain tone, while requiring high upfront engineering effort, GPU clusters, and dataset maintenance[4]. **Concrete Tool Stack:** ML engineers used the OpenAI Fine-Tuning CLI (openai api fine_tunes.create on curie and davinci), HuggingFace Seq2SeqTrainer with google/flan-t5-xl and facebook/bart-large, DeepSpeed ZeRO, and AWS SageMaker multi-GPU instances[4]. **Use Cases & Context Switching:** Enterprise ML teams at legal-tech, e-commerce, and healthcare firms built high-volume proprietary classifiers or specialized customer-support responders where passing 1,500 few-shot tokens on every Step 3 call was economically unviable[4]. **Step Resource Use:** A single task (amortizing fine-tuning compute across deployment runs) averages $\sim 1,000$ tokens, $\sim 2.0 \times 10^{15}\ \text{FLOPs}$, $\$0.25\ \text{USD}$, $30.0\ \text{Wh}$, and $60\ \text{minutes}$ of human data preparation versus $20.0\ \text{seconds}$ of amortized GPU execution, serving 5% of users while consuming 15% of tokens and 15% of era spend[4].

Full-Ladder Resource Totals (L2)

Aggregate Era Footprint (2020-06 to 2022-11): Across its 30-month window, L_2 expanded the active practitioner base to $\sim 5\ \text{million}$ users (driven by 1.2M+ early GitHub Copilot users and OpenAI Playground developers), logging $\sim 900\ \text{million}$ human hours[4]. Total era compute across training (GPT-3 at $3.14 \times 10^{23}\ \text{FLOPs}$, PaLM-540B at

$2.5 \times 10^{24}\ \text{FLOPs}$, Chinchilla) and inference reached $\sim 1.8 \times 10^{25}\ \text{FLOPs}$, consuming $\sim 320,000\ \text{MWh}$ of electricity, $\sim 45\ \text{billion}$ API and autocomplete tokens, and $\sim \$1.4\ \text{billion}$ in combined API, cloud compute, and early Copilot and SaaS spend[3].

3.3. Era 3 Ladder (L3: 2022-11 to 2023-11) — Conversational Chat, Custom RAG & Unconstrained Agent Loops

Bottom-Line Summary: Era 3 (2022-11 to 2023-11) brought generative AI to 100 million+ mainstream consumers at Step 1 while touching off a massive developer race to overcome narrow 4,096-to-8,192 token context windows using external vector databases (Step 3) and experimental autonomous loops (Step 5)[2]. **Custom Build Mania:** Because web chatbots in early 2023 could not read private corporate PDFs or execute reliable multi-step actions natively, enterprises built nearly half of their solutions internally using Python orchestration libraries (LangChain and LlamaIndex)[2]. **Brittle Autonomy Ceiling:** The era's new top step—unconstrained reason-and-act (ReAct) agent loops exemplified by AutoGPT and BabyAGI—consumed $45,000+$ tokens per run ($1.80 USD) but suffered compounding hallucination loops that prevented production deployment[4].

Valid Dates & Why Old Ladder Got Outdated

Era Window (2022-11 to 2023-11): Era 3 opened on November 30, 2022 with the launch of ChatGPT (gpt-3.5-turbo), accelerated in March 2023 with GPT-4 (8k/32k context) and Anthropic's Claude 1.3, expanded into open-weight models in July 2023 with Meta's Llama-2 (7B/13B/70B) and September 2023 with Mistral-7B, and closed in November 2023 at OpenAI DevDay[4]. **Why L2 Got Outdated:** gpt-3.5-turbo priced tokens at $2.00/1M ($10\times$ cheaper than text-davinci-003) and replaced awkward text-completion stop sequences with a multi-turn chat interface (system, user, and assistant message arrays), instantly sinking L_2's zero-shot and few-shot completion steps into accessible everyday chat[4]. **Why L3 Ended in November 2023:** In November 2023, OpenAI released GPT-4 Turbo (128k context window, native multi-function calling, JSON mode, and the Assistants/Custom GPTs file-upload API) alongside Anthropic's Claude 2.1 (200k context), which absorbed custom single-document RAG pipelines directly into Step 1 consumer chat and forced developers to abandon unconstrained AutoGPT loops for structured workflow graphs[4].

How Steps Changed

Cross-Era Step Transitions: Five structural shifts reorganized L_2 into L_3[1]. **Sinking into Web Chat:** L_2's Step 2 (Zero-Shot Completion) and basic Step 3 (Few-Shot Prompting) sunk into **Step 1 (Consumer Web Chat & Conversational Assistance, L3-P1)**, where non-technical office workers iterated via multi-turn browser dialogue[1]. **Template Chaining Evolution:** L_2's

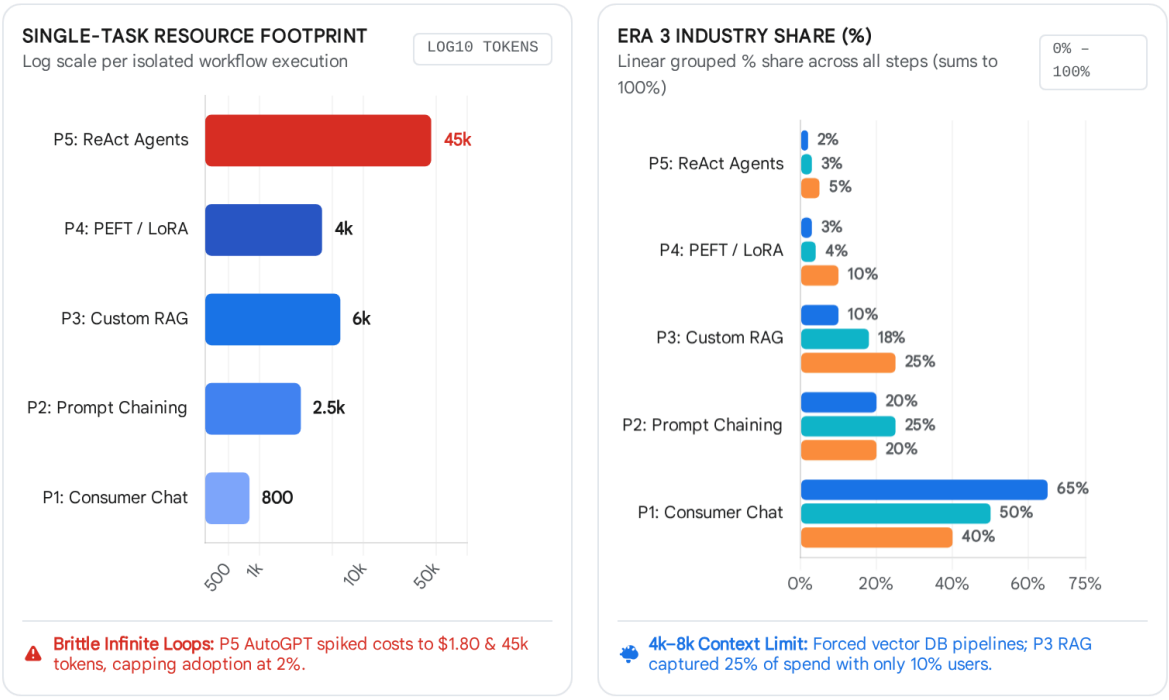
complex in-context chains formalized into **Structured Prompt Engineering & Template Chaining (L3-P2)** using programmatic system prompts, output parsers, and sequential chains[4]. **Birth of Custom RAG: Custom Retrieval-Augmented Generation (L3-P3)** emerged as the signature enterprise engineering tier to bypass 4k–8k context limits by embedding private text chunks into vector databases[4]. **Adapter Fine-Tuning Shift:** Full-weight fine-tuning transformed into **Parameter-Efficient Fine-Tuning (PEFT / LoRA, L3-P4)** on open-weight Llama-2 and Mistral models using 4-bit quantization (QLoRA)[4]. **New Agent Apex: Autonomous ReAct Loops & Unconstrained Agents (L3-P5)** appeared at the top of the ladder (AutoGPT, BabyAGI), testing whether an LLM could loop indefinitely over goals without human intervention[4].

ERA 3 LADDER (2022-11 TO 2023-11) The Modern LLM & RAG Explosion

Era 3 Custom RAG Captured Enterprise Spend While Early AutoGPT Loops Spiked Single-Task Costs 1,000x

Left Panel Metric: Tokens | Cost ($) | Energy (Wh) | FLOPs

User % Token % Spend %



P3: Custom Retrieval-Augmented Generation (RAG)

P3

Workaround for 4k-8k context limits; became enterprise workhorse driving 25% of all spending with 10% users.

`Tokens: 6,000 • USD: $0.12 • Energy: 15 Wh | Users: 10% • Spend: 25%`

In 2023, limited 4k–8k context windows forced developers to build custom vector-database pipelines for document search, while unconstrained agent loops burned massive token budgets. Although 65% of users stayed inside default browser chat at Step 1, Step 3 custom RAG pipelines accounted for 25% of industry spending, and Step 5 ReAct loops required 45,000 tokens ($1.80 and 180 Wh) per attempt.

Data sources: Menlo Ventures, Sequoia Capital, Luccioni et al. (2022), PricePerToken, DeepLearning.AI, TinkerLLM

Step-by-Step Breakdown (Steps 1 to 5)

Step 1 — Consumer Web Chat & Conversational Assistance (L3-P1): Step Boundaries: Step 1 is the baseline interactive dialogue tier (~800 tokens per turn) inside a browser window, requiring zero coding or API keys unlike Step 2 programmatic integration, but blocked from reading private multi-document repositories (requiring Step 3 RAG) due to 4,096–8,192 token

context cutoffs and a lack of native file-upload parsers in early 2023[1]. **Concrete Tool Stack:** Everyday users accessed the ChatGPT free web UI (gpt-3.5-turbo) and ChatGPT Plus ($20/month GPT-4 8k), Anthropic Claude 1.3 and Claude 2.0 web chat, Microsoft Bing Chat (Sydney), and Google Bard[1]. **Use Cases & Context Switching:** Knowledge workers drafted emails, students summarized pasted article excerpts, and software developers copy-pasted stack traces into a browser tab (documented in the 2023 Stack Overflow Developer Survey and DeepMind's placement of ChatGPT and Bard at Emerging AGI)[22]; a developer used Step 1 for ad-hoc syntax questions before switching to Step 2 or Step 3 when building an automated software feature[4]. **Step Resource Use:** A single task uses ~ 800 tokens, $\sim 3.2 \times 10^{13}\ \text{FLOPs}$, $\$0.0016\ \text{USD}$ (at the gpt-3.5-turbo $2/1M rate), $0.3\ \text{Wh}$, and $2\ \text{minutes}$ of human typing and reading versus $2.5\ \text{seconds}$ of AI streaming, capturing 65% of era users, 50% of global token traffic, and 40% of era spend[4].

Step 2 — Structured Prompt Engineering & Template Chaining (L3-P2): Step Boundaries: Step 2 moves from manual browser chat to programmatic API pipelines that chain 2 to 4 sequential LLM calls (~2,500 tokens total) using structured system instructions, few-shot templates, and output moderation checks, differing from Step 3 by relying strictly on data passed directly in the prompt rather than external vector retrieval[4]. **Concrete Tool Stack:** Engineers wired the OpenAI ChatCompletion API (gpt-3.5-turbo-0613, gpt-4-0613), LangChain (PromptTemplate, LLMChain, SimpleSequentialChain, StructuredOutputParser), and Gradio for rapid UI demos[4]. **Use Cases & Context Switching:** These patterns were taught to millions of engineers in Andrew Ng and Isa Fulford's landmark May 2023 courses *ChatGPT Prompt Engineering for Developers* and *Building Systems with ChatGPT* (implementing classification-then-response chains and moderation guardrails)[17]; engineers used Step 2 for deterministic text processing and switched to Step 3 whenever reference documents exceeded 3,000 words[4]. **Step Resource Use:** A single task processes $\sim 2,500$ tokens across chained calls, $\sim 1.0 \times 10^{14}\ \text{FLOPs}$, $\$0.005\ \text{USD}$ (on gpt-3.5-turbo), $1.0\ \text{Wh}$, and $8\ \text{minutes}$ of human prompt debugging versus $6.0\ \text{seconds}$ of AI execution, accounting for 20% of users, 25% of tokens, and 20% of era spend[4].

Step 3 — Custom Retrieval-Augmented Generation (L3-P3): Step Boundaries: Step 3 splits private documents (PDFs, Notion wikis, SQL tables) into 500-token chunks, converts them into numerical vectors (embeddings), stores them in a vector database, and injects the top-k matching chunks (~6,000 tokens across retrieval and synthesis) into an LLM prompt, solving Step 1 and Step 2's memory cutoff and hallucination on private data, while failing when questions require synthesizing an entire 500-page book or executing live actions in external software[4]. **Concrete Tool Stack:** Developers combined LangChain (DocumentLoaders, RecursiveCharacterTextSplitter, RetrievalQA), LlamaIndex (created by Jerry Liu), OpenAI text-embedding-ada-002 or Cohere Embeddings, and vector databases Pinecone, Chroma,

Weaviate, Qdrant, and FAISS paired with GPT-4[4]. **Use Cases & Context Switching:** Custom RAG was the #1 enterprise engineering pattern in Menlo Ventures' 2023 report ($1.7B total market) and the centerpiece of Harrison Chase's *LangChain: Chat with Your Data* and Jerry Liu's *Building and Evaluating Advanced RAG* courses on DeepLearning.AI[2]; companies built internal "Chat with your corporate PDFs" portals at Step 3 while using Step 1 for general brainstorming[4]. **Step Resource Use:** A single task processes $\sim 6,000$ tokens (embedding + top-k context + GPT-4 answer), $\sim 1.5 \times 10^{15}\ \text{FLOPs}$, $\$0.12\ \text{USD}$ (at GPT-4 30/60 per 1M prices), $15.0\ \text{Wh}$ [19], and $15\ \text{minutes}$ of human chunking and retrieval tuning versus $12.0\ \text{seconds}$ of pipeline latency, serving 10% of users while generating 18% of token volume and 25% of era spend[4].

Step 4 — Parameter-Efficient Fine-Tuning (PEFT / LoRA, L3-P4): Step Boundaries: Step 4 freezes the base weights of an open-weight 7B–70B model (Llama-2 or Mistral-7B) and trains tiny low-rank adapter matrices (LoRA) in 4-bit precision (QLoRA) on a single 24GB–80GB GPU, providing full data privacy, custom domain formatting, and low inference cost compared to Step 3 GPT-4 calls, but unable to dynamically fetch real-time facts without still pairing with a Step 3 RAG pipeline[4]. **Concrete Tool Stack:** Open-source practitioners deployed Meta Llama-2-7b-hf and 70b, Mistral-7B-v0.1, HuggingFace peft, bitsandbytes (Tim Dettmers' 4-bit NF4), trl (SFTTrainer), Axolotl, and vLLM for high-throughput serving on AWS or RunPod A100 GPUs[3]. **Use Cases & Context Switching:** Healthcare, defense, and financial engineering teams prohibited from sending customer data to OpenAI APIs—plus open-source developers training specialized SQL-generation or medical-coding adapters—switched to Step 4 whenever Step 3 API costs or privacy compliance blocked deployment[2]. **Step Resource Use:** A single task (amortizing a 2-hour $5 QLoRA training run over deployment queries) processes $\sim 4,000$ tokens, $\sim 5.6 \times 10^{14}\ \text{FLOPs}$, $\$0.04\ \text{USD}$, $8.0\ \text{Wh}$, and $45\ \text{minutes}$ of dataset formatting and evaluation versus $15.0\ \text{seconds}$ of amortized GPU runtime, representing 3% of users, 4% of tokens, and 10% of era spend[4].

Step 5 — Autonomous ReAct Loops & Unconstrained Agents (L3-P5): Step Boundaries: Step 5 was Era 3's experimental apex—wrapping GPT-4 inside an open-ended while loop that prompted the model to generate a Thought, select an Action (web search, Python REPL, file write), read the Observation (ReAct pattern), and repeat 15 to 50 times (~45,000 tokens) until a high-level goal was met—differing from Steps 1–4 by granting the model autonomous control flow, but lacking reliable tool schemas and state checkpoints, which caused infinite loops and context overflow[4]. **Concrete Tool Stack:** Early adopters experimented with AutoGPT (Significant Gravitas, the fastest-growing GitHub repository of Spring 2023), BabyAGI (Yohei Nakajima), AgentGPT, and LangChain AgentExecutor (zero-shot-react-description)[4]. **Use Cases & Context Switching:** Viral developer demos attempted to "conduct autonomous market research and build a website"; as LangChain CEO Harrison Chase later documented

("The first autonomous agents didn't work, why?"), a 90% per-step reliability rate compounded over 20 unconstrained steps drops to $0.90^{20} = 12.2\%$ success, forcing developers to abandon Step 5 after burning $50 in API credits and retreat to Step 2 and Step 3 chains[4]. **Step Resource Use:** A single task consumes $\sim 45,000$ tokens across recursive GPT-4 context re-reads, $\sim 2.5 \times 10^{16}\ \text{FLOPs}$, $\$1.80\ \text{USD}$, $180.0\ \text{Wh}$, and $25\ \text{minutes}$ of human babysitting and aborting loops versus $120.0\ \text{seconds}$ (2 minutes) of AI runtime, representing 2% of users, 3% of tokens, and 5% of era spend[4].

Full-Ladder Resource Totals (L3)

Aggregate Era Footprint (2022-11 to 2023-11): Across its 12-month window, L_3 exploded to $\sim 180\ \text{million}$ active users globally, logging $\sim 6.5\ \text{billion}$ human hours[4]. Total era compute across frontier training (GPT-4 at $\sim 2 \times 10^{25}\ \text{FLOPs}$, Llama-2, Claude 2) and massive consumer and API inference reached $\sim 3.5 \times 10^{26}\ \text{FLOPs}$, consuming $\sim 2.4\ \text{million MWh}$ (2.4 TWh) of electricity, $\sim 1.8\ \text{trillion}$ tokens, and $\sim \$4.5\ \text{billion}$ in combined enterprise ($1.7B per Menlo Ventures) and consumer subscription spend[2].

3.4. Era 4 Ladder (L4: 2023-11 to 2024-09) — Multimodal Context, Native Tool-Calling & Scaffolded Harnesses

Bottom-Line Summary: Era 4 (2023-11 to 2024-09) dismantled Era 3's custom engineering bottlenecks through three breakthroughs: 128k-to-2M token context windows with 50%–90% prompt caching discounts, native multimodal vision and structured JSON tool-calling, and the release of Claude 3.5 Sonnet (June 2024) which ignited multi-file AI code editors (Cursor)[4]. **RAG Sinks into Chat UI:** Simple single-document vector RAG (L3-P3) sunk directly into Step 1's paperclip file-upload button in ChatGPT Plus and Claude.ai[4]. **State Harnesses Replace Unconstrained Loops:** Having learned from 2023 AutoGPT failures, developers split multi-step execution into Step 2 predefined workflows (routing, chaining, parallelization) and Step 5 cyclic state-machine harnesses (LangGraph, SWE-agent)[4].

Valid Dates & Why Old Ladder Got Outdated

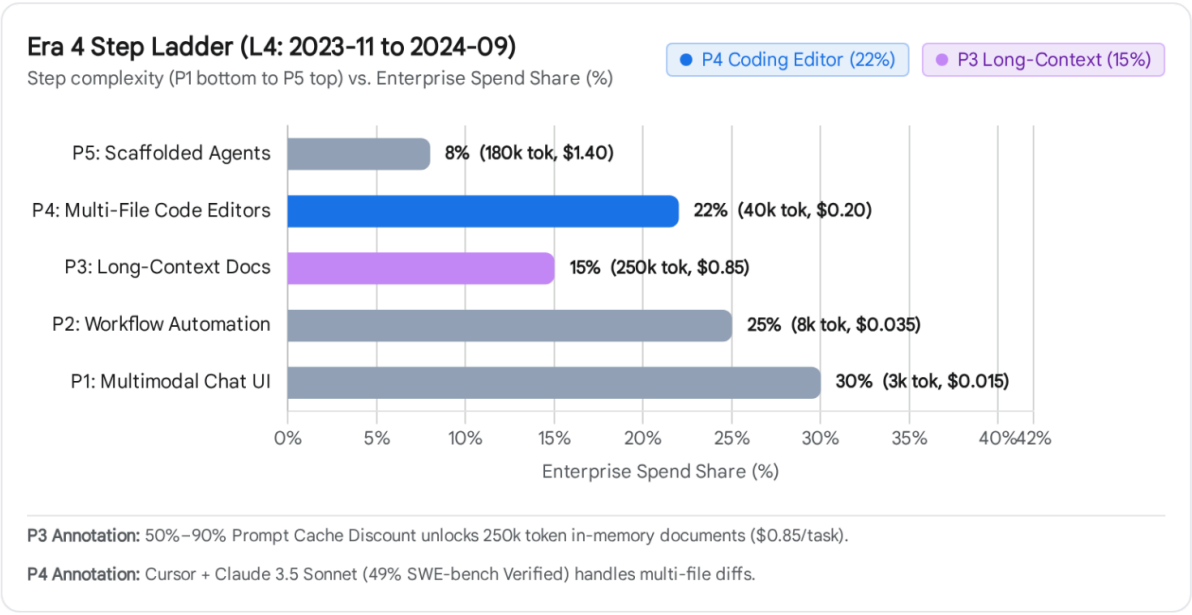
Era Window (2023-11 to 2024-09): Era 4 launched in November 2023 with OpenAI's GPT-4 Turbo (128k context, $10/30$ per 1M tokens—a $3\times$ price cut) and Anthropic's Claude 2.1 (200k context), accelerated in February–May 2024 with Google's Gemini 1.5 Pro (1M–2M token context window) and OpenAI's omni-modal GPT-4o ($5/15$ per 1M), peaked in June 2024 with Anthropic's Claude 3.5 Sonnet ($3/15$ per 1M, jumping from 22% on Claude 3 Opus to 33% on SWE-bench Verified) plus interactive Claude Artifacts, and concluded in September 2024 when

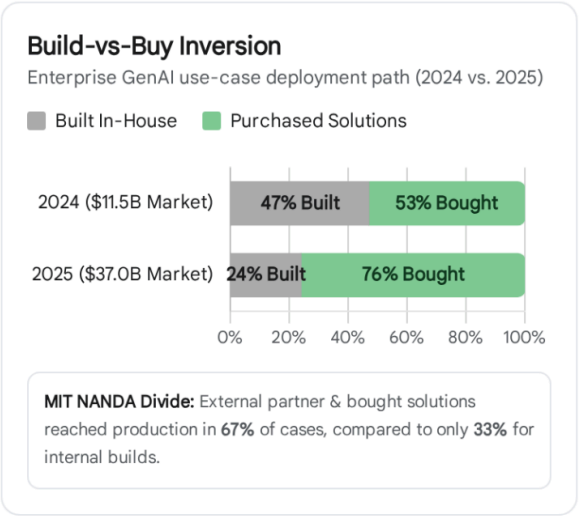
OpenAI introduced test-time reasoning scaling with o1-preview[6]. **Why L3 Got Outdated:** When any consumer could drag-and-drop a 150-page PDF or screenshot directly into a 200k-context chat window and when prompt caching cut repeated context costs by up to 90% on Anthropic and 50% on OpenAI, maintaining custom Python chunking scripts and Pinecone indexes for small-to-medium document sets became unnecessary overhead[4]. Simultaneously, Menlo Ventures documented enterprise GenAI spending surging from $1.7B in 2023 to $11.5B$–13.8B in 2024 as pilots transitioned into production[2].

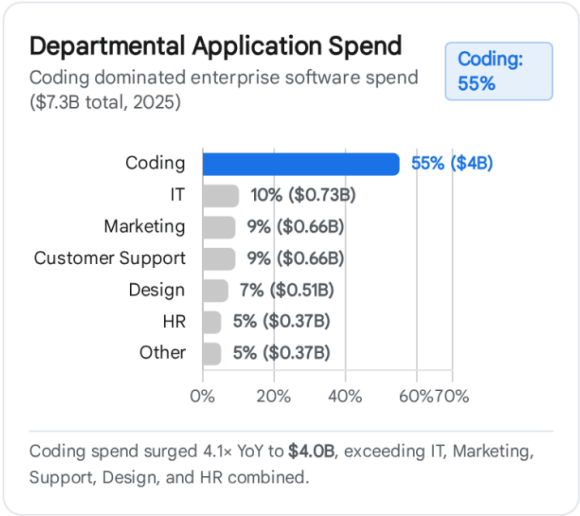
How Steps Changed

Cross-Era Step Transitions: Five major step realignments separated L_4 from L_3[4]. **Basic RAG Sinks into Default UI:** L_3's Step 3 (basic single-document and multi-PDF RAG) and reading text from images (OCR-to-LLM) sunk into **Step 1 (Multimodal Chat UI with Built-In Retrieval, L4-P1)** via native file uploads, vision inputs, and Claude Artifacts[4]. **Predefined Workflow Consolidation:** L_3's prompt chains and single-tool routers merged with native API JSON schema enforcement into **Deterministic Workflow Automation & Native Tool Calling (L4-P2)**—the exact tier Anthropic codified as predefined Workflows[4]. **In-Memory Long Context: Long-Context In-Memory Document Analysis (L4-P3)** replaced chunked RAG for 100k–2M token corpora by stuffing entire codebases, legal discovery bundles, or video transcripts into KV-cache memory[4]. **Multi-File IDE Coding Split:** AI coding split out of generic chat and line autocomplete into a dedicated high-growth tier: **Multi-File AI Code Editors & Inline Diffs (L4-P4)**, led by Cursor and Claude 3.5 Sonnet[4]. **State-Machine Agent Harnesses:** L_3's broken AutoGPT loops were discarded and replaced by **Scaffolded Agent Workflows & Verified Harnesses (L4-P5)**, using cyclic graph state machines (LangGraph) and unit-test feedback harnesses (SWE-agent)[4].

Long Context and Multi-File Code Editors Reshaped Era 4 as Enterprises Pivoted From Building to Buying







In 2024, 200k-to-2M token context windows and Claude 3.5 Sonnet inside Cursor drove software coding to 55% of all departmental AI spending. As basic document RAG sank into default Step 1 chat uploads and in-house pilots struggled to show returns, enterprises shifted from 47% internal custom builds in 2024 to 76% purchased ready-made applications by 2025.

Data sources: Menlo Ventures Enterprise GenAI Report, MIT NANDA & XCope Research, Anthropic Research, Introl LLM Infrastructure Guide, State of AI Report.

Step-by-Step Breakdown (Steps 1 to 5)

Step 1 — Multimodal Chat UI with Built-In Retrieval (L4-P1): Step Boundaries: Step 1 provides consumer and enterprise workspace chat (~3,000 tokens per turn including uploaded files or images) that natively ingests PDFs, spreadsheets, screenshots, and voice while rendering interactive UI previews (Artifacts), absorbing L_3's basic single-file RAG while remaining human-driven one turn at a time without automated API triggers or multi-file repository access[4]. **Concrete Tool Stack:** Users worked inside ChatGPT Plus and Enterprise (GPT-4o, Custom GPTs, Built-in Code Interpreter / Data Analysis), Claude.ai (Claude 3.5 Sonnet 20240620 with 200k Project Knowledge and Artifacts), and Google Gemini Advanced[4]. **Use Cases & Context Switching:** Microsoft's 2024 Work Trend Index (31,000 workers across 31 countries) found 75% of knowledge workers using AI at work, primarily uploading financial CSVs, slide decks, or whiteboard photos into Step 1 chat[40]; a product manager used Step 1 Claude Artifacts to prototype a React dashboard in 3 minutes before handing the code to an engineer at Step 4[4]. **Step Resource Use:** A single task processes $\sim 3,000$ tokens (prompt + uploaded document or image tokens + response), $\sim 1.2 \times 10^{14}\ \text{FLOPs}$, $\$0.015\ \text{USD}$ (at GPT-4o and Sonnet 3.5 rates), $0.5\ \text{Wh}$, and $3\ \text{minutes}$ of human interaction versus $4.0\ \text{seconds}$ of AI generation, capturing 55% of active users, 35% of token volume, and 30% of era spend[4].

Step 2 — Deterministic Workflow Automation & Native Tool Calling (L4-P2): Step Boundaries: Step 2 comprises programmatic backend pipelines (~8,000 tokens) where predefined code orchestrates LLM calls via structured JSON outputs, routing, prompt chaining, or parallel sectioning and voting[7], differing from Step 1 by running headless inside production software and from Step 5 because fixed application code—not the LLM—controls the execution path (Anthropic's exact definition of a Workflow)[7]. **Concrete Tool Stack:** Backend teams deployed OpenAI Structured Outputs (strict JSON Schema adherence), Anthropic Tool Use API, Pydantic and Instructor (by Jason Liu), LangChain Expression Language (LCEL), and hybrid keyword-plus-vector search (BM25 + dense embeddings with Cohere Rerank)[5]. **Use Cases & Context Switching:** Production deployments focused on high-volume customer support systems with a strict budget of ~$0.10 (30,000–50,000 tokens maximum) per ticket, automated guardrails screening queries in parallel, and multi-index production RAG taught in Anthropic's Claude API curriculum[5]; engineering teams used Step 2 workflows whenever a business decision tree could be mapped explicitly, reserving Step 5 agents only for ambiguous tasks[7]. **Step Resource Use:** A single task processes $\sim 8,000$ tokens across 3–5 chained or parallel calls, $\sim 4.0 \times 10^{14}\ \text{FLOPs}$, $\$0.035\ \text{USD}$, $1.8\ \text{Wh}$, and $5\ \text{minutes}$ of amortized human monitoring versus $8.0\ \text{seconds}$ of pipeline runtime, accounting for 22% of users, 25% of tokens, and 25% of era spend[4].

Step 3 — Long-Context In-Memory Document Analysis (L4-P3): Step Boundaries: Step 3 loads 100,000 to 2,000,000 tokens (entire books, 50-page financial filings, multi-hour video

recordings, or whole codebases) directly into the model's active context memory (KV-cache) rather than chunking via vector search, eliminating Step 2 and Step 3 retrieval loss when reasoning across distant chapters, while incurring high first-turn prefill latency (20–40 seconds) and requiring prompt caching (minimum 1,024 tokens) to avoid paying full input prices on follow-up turns[4]. **Concrete Tool Stack:** Practitioners utilized Google Gemini 1.5 Pro (1M–2M context window on Vertex AI), Anthropic Claude 3.5 Sonnet and Claude 3 Haiku with Anthropic Prompt Caching (90% cost reduction and 85% latency reduction on cached prefixes), and OpenAI Prompt Caching (automatic 50% discount)[55]. **Use Cases & Context Switching:** Legal discovery teams cross-checked depositions against 500 pages of exhibits, equity analysts queried 10-K filings, and healthcare vertical AI systems (which grew toward $450M in 2024 on the way to $1.5B in 2025 per Menlo Ventures) synthesized complete longitudinal patient medical records[2]. **Step Resource Use:** A single task processes $\sim 250,000$ tokens, $\sim 1.5 \times 10^{16}$ FLOPs (including attention prefill), $\$0.85$ USD (blending uncached prefill at $3/1M with cached follow-up turns at $0.30/1M), 35.0 Wh, and 10 minutes of human analysis versus 30.0 seconds of AI prefill and generation, representing 8% of users, 20% of global token volume, and 15% of era spend[4].

Step 4 — Multi-File AI Code Editors & Inline Diffs (L4-P4): Step Boundaries: Step 4 integrates frontier models (Claude 3.5 Sonnet) directly into an AI-native code editor (IDE) capable of indexing the local codebase, reading multiple referenced files (@file, @codebase), and proposing synchronized multi-file code diffs under human "plan, approve, execute" supervision (SAE Level 3)[4], far exceeding L_2's single-line Copilot autocomplete while stopping short of autonomous background terminal and test execution (Step 5)[11]. **Concrete Tool Stack:** Software teams adopted the Cursor IDE (Cmd+K inline edit, Cmd+L codebase chat, early Composer multi-file diff), Zed AI editor, and GitHub Copilot Chat / Workspace, powered predominantly by claude-3-5-sonnet-20240620 ($3/15 per 1M tokens, 200k context) and GPT-4o[6]. **Use Cases & Context Switching:** Software engineers refactored React components across 5 files, wrote API endpoints with matching database schemas, and debugged compiler errors inside Cursor (driving the surge in coding AI spend documented by Menlo Ventures)[2]; developers used fast Tab predictions for single lines, Cmd+K for single functions, and Step 4 multi-file Composer for feature implementation[4]. **Step Resource Use:** A single multi-file edit session processes $\sim 40,000$ tokens, $\sim 2.0 \times 10^{15}$ FLOPs, $\$0.20$ USD, 8.0 Wh, and 4 minutes of human diff review and approval versus 15.0 seconds of AI generation, capturing 12% of users, 15% of tokens, and 22% of era spend[4].

Step 5 — Scaffolded Agent Workflows & Verified Harnesses (L4-P5): Step Boundaries: Step 5 is the highest complexity tier of Era 4—placing an LLM inside a cyclic state-machine graph or sandboxed software engineering harness where the model plans subtasks, executes bash and search tools, runs automated unit tests, reads error tracebacks, and iterates across 10 to 50 turns (~180,000 tokens)[4]—solving L_3 AutoGPT divergence by constraining state

transitions and using objective test verification, though capped at 33%–45% success on SWE-bench Verified prior to late October 2024[6]. **Concrete Tool Stack:** Agent architects built with LangGraph (cyclic graph state machines with human-in-the-loop checkpoints), CrewAI (role-based multi-agent teams), AutoGen (Microsoft), Princeton's SWE-agent, and Cognition's Devin (March 2024 preview) running Claude 3.5 Sonnet or GPT-4o[4]. **Use Cases & Context Switching:** Teams targeted automated resolution of real GitHub issues in sandboxed Docker containers (SWE-bench), complex multi-source market intelligence synthesis (Anthropic's Orchestrator-Workers and Evaluator-Optimizer patterns), and workflows taught in DeepLearning.AI's *Multi AI Agent Systems with crewAI*[7]; engineers delegated well-tested bug tickets to Step 5 harnesses while keeping ambiguous architecture changes in Step 4 Cursor[11].

Step Resource Use: A single task consumes $\sim 180,000$ tokens across iterative test-and-fix loops, $\sim 1.2 \times 10^{16}\ \text{FLOPs}$, $\$1.40\ \text{USD}$, $50.0\ \text{Wh}$, and $15\ \text{minutes}$ of human task scoping and PR review versus $180.0\ \text{seconds}$ (3 minutes) of autonomous runtime, representing 3% of users, 5% of tokens, and 8% of era spend[4].

Full-Ladder Resource Totals (L4)

Aggregate Era Footprint (2023-11 to 2024-09): Across its 10-month window, L_4 scaled to $\sim 350\ \text{million}$ active users globally, logging $\sim 14\ \text{billion}$ human hours[4]. Total era compute across training (GPT-4o at 10^{25} _ $5 \times 10^{25}\ \text{FLOPs}$, Claude 3.5 Sonnet, Llama-3.1-405B at $3.8 \times 10^{25}\ \text{FLOPs}$) and deployment inference reached $\sim 1.9 \times 10^{27}\ \text{FLOPs}$, consuming $\sim 9.5\ \text{million MWh}$ (9.5 TWh) of electricity, $\sim 15\ \text{trillion}$ tokens, and $\sim \$14.5\ \text{billion}$ in combined enterprise ($11.5B$–13.8B annualized per Menlo Ventures) and consumer spend[2].

3.5. Era 5 Ladder (L5: 2024-09 to 2026-10) — Test-Time Compute Reasoning, Protocol Standardization & Autonomous Agents

Bottom-Line Summary: Era 5 (2024-09 to 2026-10) marks the transition from conversational assistants to autonomous digital workers, driven by three structural pillars: training models to think step by step using verifiable rewards (RLVR test-time compute scaling), standardizing external tool connections via the open Model Context Protocol (MCP), and deploying autonomous command-line (Claude Code) and graphical desktop (Computer Use) agents[5]. **Benchmark Progression & Generalization:** Between October 2024 and September 2026, coding agent resolution on SWE-bench Verified climbed from 49% (Claude 3.5 Sonnet v2) to 72.7% (Claude Sonnet 4), 79.6% (Claude Sonnet 4.6), and 85.8%–96.0% on frontier harnesses, prompting the industry to migrate toward the harder 1,865-task SWE-bench Pro benchmark where Muse Spark 1.1 (61.5% public / 51.5% private commercial) and Claude Opus 5.5 (89.9% vendor scaffold) lead[6]. **Macro Spend Explosion:** Enterprise generative AI spending hit $37.0

billion in 2025 (6% of the entire global SaaS market), while consumer AI spending tripled in 2026[2].

Valid Dates & Why Old Ladder Got Outdated

Era Window (2024-09 to 2026-10): Era 5 began with three consecutive breakthroughs in Autumn 2024: (1) OpenAI's September 2024 release of o1-preview and o1-mini (followed by o3 in December 2024 and GPT-5 / GPT-5.4 in 2025–2026), which introduced extended internal thinking chains trained via reinforcement learning on verifiable math and code rewards (RLVR)[8]; (2) Anthropic's October 22, 2024 release of upgraded Claude 3.5 Sonnet (20241022) achieving 49% on SWE-bench Verified and introducing beta Computer Use (screen-reading and cursor-clicking) alongside Claude Code[5]; and (3) Anthropic's November 26, 2024 release of the Model Context Protocol (MCP) as an open standard for connecting AI systems to external tools and data[58]. **Why L4 Got Outdated:** Pre-September 2024 models generated tokens at a fixed, shallow compute depth per output word and required bespoke "N×M" custom integration code for every model-to-tool pair[9]. Once models could dynamically spend 30 seconds to 15 minutes of internal test-time compute (pass@k and extended thinking) and plug into any enterprise database via a universal MCP server—donated by Anthropic in December 2025 to the Linux Foundation's Agentic AI Foundation (AAIF), co-founded with Block and OpenAI and backed by Google, Microsoft, AWS, Cloudflare, and Bloomberg— L_4 's isolated prompt-engineering and bespoke API glue became outdated[8].

How Steps Changed

Cross-Era Step Transitions: Five structural shifts transformed L_4 into L_5[4]. **Interactive Artifacts Sink into Step 1:** L_4 's multimodal analysis, voice conversation, live web search, and interactive UI dashboards (MCP Apps and Artifacts) sunk into **Step 1 (Conversational Artifact & Knowledge Interaction, L5-P1)**[4]. **Birth of Step 2 Reasoners:** OpenAI's Level 2 ("Reasoners") materialized as **Test-Time Compute Reasoning & Deep Research (L5-P2)**, where users hand over complex math, strategic planning, and multi-source literature synthesis to extended-thinking models (o1, o3, GPT-5.4 xHigh, Claude Opus 4.6 thinking)[1]. **MCP Protocol Consolidation:** L_4 's custom tool-calling and RAG pipelines consolidated into **Standardized Protocol Tool & Data Integration (MCP, L5-P3)**, collapsing the $N \times M$ integration problem into $N + M$ reusable MCP clients and servers (surpassing 10,000 public servers and 97 million monthly SDK downloads by December 2025)[4]. **Autonomous CLI Coding Elevation:** L_4 's inline IDE diffs evolved into **Autonomous Terminal & IDE Coding Agents (L5-P4)** (Claude Code, Cursor Composer 2, Mini-SWE-agent, GPT-5.2 Codex), where agents autonomously run bash, grep, glob, and edit loops across whole repositories with 64,000-token outputs and 1M-token context windows[4]. **Multi-Agent & GUI Apex: Full-System Multi-Agent Workflows & Computer Use (L5-P5)** reached the top of the ladder, combining orchestrator-worker subagent swarms (LangChain Deep Agents, Claude Opus 5.5) with native desktop and browser

graphical screen control (OSWorld-Verified reaching 72.5%–78.4%)[7].

Benchmark Saturation vs. Private Code Generalization: Empirical evaluation data through September 2026 reveals a critical divergence between public benchmark saturation and unseen commercial software engineering[6]. While SWE-bench Verified climbed from 22.0% on Claude 3 Opus (2024-03) and 33.0% on Claude 3.5 Sonnet v1 (2024-06) to 49.0% (Claude 3.5 Sonnet v2, 2024-10), 72.7% (Claude Sonnet 4, 2025-05), 78.8% (Gemini 3.5 Flash), 79.6% (Claude Sonnet 4.6, 2026-02), 85.8% (Claude Code / Mini-SWE-agent), and 96.0% (Claude Opus 5 / 5.5 vendor aggregate), training data contamination and custom vendor scaffolding prompted Scale AI to establish SWE-bench Pro across 1,865 tasks in 41 repositories (731 public tasks and 276 unseen private commercial tasks averaging 107.4 lines changed across Python, Go, TypeScript, and JavaScript)[6]. On Scale AI's standardized SWE-bench Pro harness (checked September 2026), two crucial gaps emerge: first, vendor-reported scaffolds score 10 to 30 percentage points higher than standardized harnesses (for example, GPT-5.2 Codex reports 56.4% on its own scaffold versus 41.0% under Scale's standardized harness, a 15.4-point scaffold gap, while Claude Opus 5.5 leads vendor-reported aggregates at 89.9% at $20/M output ahead of Claude Fable 5 at 80.0% at $50/M)[10]; second, every frontier model suffers a 4.8 to 22.5 percentage-point drop when moving from public repositories to unseen private commercial codebases (Muse Spark 1.1 by Meta leads both boards but drops 10.0 points from 61.5% public to 51.5% commercial; Claude Opus 4.6 thinking drops 4.8 points from 51.9% to 47.1%; GPT-5.4 xHigh drops 15.7 points from 59.1% to 43.4%; and Claude Opus 4.5 drops 22.5 points from 45.9% public to 23.4% commercial)[10].

Model / Agent System (Release / Audit Date)	**SWE-bench Verified Score (%)**	**SWE-bench Pro Public Set (731 Tasks, Pass@1)**	**SWE-bench Pro Unseen Commercial Set (276 Tasks)**	**Public-to-Private Generalization Gap / Notes**
Claude 3 Opus (2024-03)[13]	22.0%	Not evaluated	Not evaluated	Pre-Sonnet 3.5 baseline under Anthropic scaffold[13].
Claude 3.5 Sonnet v1 (2024-06)[6]	33.0%	Not evaluated	Not evaluated	$3/15$ per 1M tokens; 200k context window[6].
Claude 3.5 Sonnet v2 (2024-10)[6]	49.0% (53% MIT index[71])	Not evaluated	Not evaluated	Introduced Computer Use &

				bash/str_replace scaffold[13].
Claude Sonnet 4 (2025-05)[6]	72.7%	42.7% ±	Not disclosed	Added 64k output ceiling & 1M beta context (Aug 2025)[6].
Claude Opus 4.5 (2025-11)[10]	Not disclosed	45.9%	23.4%	−22.5 pt drop on unseen private commercial code[10].
GPT-5.2 Codex (2026)[10]	Not disclosed	41.0% ± (Scale)	Not disclosed	+15.4 pt scaff (56.4% on vendor scaffold)[10].
Claude Sonnet 4.6 (2026-02)[6]	77.4%–79.6%	Not published	Not published	72.5% OSWorld-Verified; 59.1% Terminal-Bench 2.1[49].
Gemini 3.5 Flash (2026)[49]	78.8% (contaminated)	55.1%	Not published	78.4% OSWorld; 76.2% Terminal-Bench; 83.6% MCP Atlas[49].
Claude Opus 4.6 (thinking) (2026)[10]	Not disclosed	51.9% ±	47.1% ±	−4.8 pt drop; #2 on commercial unseen code (mini-swe-agent)[10].

GPT-5.4 (xHigh) (2026)[10]	Not disclosed	$59.1\% \pm$	$43.4\% \pm$	-15.7 pt drop; $2.50/M pricing; 1.1M context[10].
Muse Spark 1.1 (Meta) (2026-09)[10]	Not disclosed	$61.5\% \pm$ (#1)	$51.5\% \pm$ (#1)	-10.0 pt drop; tops both Scale standardized sets[10].
Claude Opus 5.5 (2026-09)[10]	96.0% (Opus 5)[57]	89.9% (Vendor agg.)	Not disclosed	#1 on llm-stats vendor aggregate at $20/M output[10].

Step-by-Step Breakdown (Steps 1 to 5)

Step 1 — Conversational Artifact & Knowledge Interaction (L5-P1): Step Boundaries: Step 1 is the universal multimodal workspace (~4,000 tokens per turn) featuring instant voice, camera vision, live web search, persistent user memory, and interactive mini-applications rendered directly inside chat (Artifacts and MCP Apps)[4], serving as the default consumer and office entrypoint whereas Step 2 invokes extended multi-step reasoning chains before answering[1]. **Concrete Tool Stack:** Users interact through ChatGPT (GPT-4o and GPT-5.4 standard mode), Claude.ai (Claude Sonnet 4.6 and Claude Haiku 4.5 with MCP Apps interactive dashboards and forms), and Google Gemini 3.5 Flash[6]. **Use Cases & Context Switching:** Used weekly by 87% of US workers at large employers (KPMG mid-2025 survey) and 37.4% of the total US workforce (St. Louis Fed) for drafting, interactive spreadsheet rendering, and daily Q&A[12]; knowledge workers stay on Step 1 for rapid 3-second answers and toggle "Extended Thinking" or "Deep Research" (Step 2) when tackling complex analysis[1]. **Step Resource Use:** A single task processes $\sim 4,000$ tokens, $\sim 8.0 \times 10^{13}$ FLOPs (benefiting from sparse MoE architectures and Flash-tier distillation), $\$0.008$ USD, 0.3 Wh, and 3 minutes of human interaction versus 3.0 seconds of AI response, capturing 45% of active users, 20% of global token volume, and 18% of era spend[3].

Step 2 — Test-Time Compute Reasoning & Deep Research (L5-P2): Step Boundaries: Step 2 scales compute during inference (test-time compute) by training the model via reinforcement learning on verifiable rewards (RLVR) to generate thousands of internal reasoning tokens—verifying hypotheses, backtracking from dead ends, or browsing dozens of web sources (~35,000 tokens)—before emitting a final answer[4], corresponding directly to OpenAI

AGI Level 2 ("Reasoners")[1] and differing from Step 3 and Step 4 by focusing on analytical synthesis and formal verification rather than mutating external enterprise databases or code repositories[1]. **Concrete Tool Stack:** Practitioners invoke OpenAI o1, o3, GPT-5 (High), GPT-5.4 (xHigh), Claude Opus 4.6 (thinking), Claude Sonnet 4 and 4.6 extended thinking mode, and Deep Research autonomous literature synthesis agents[6]. **Use Cases & Context Switching:** Analysts and scientists apply Step 2 to research-level mathematics (AIME), graduate-level science (GPQA Diamond where Sonnet 4 hit 70.0% and GPT-6 Astra reached 96%), tax and legal strategy, and competitive market dossiers taught in DeepLearning.AI's *Reasoning with o1*[1]; as Epoch AI documented, spending 2 to 6 orders of magnitude more inference compute (pass@k and sequential thinking) allows models to solve formal math and logic tasks that would otherwise require $1,000\times$ larger pre-training runs[8]. **Step Resource Use:** A single task processes $\sim 35,000$ tokens (primarily hidden or expandable thinking tokens plus search synthesis), $\sim 2.8 \times 10^{15}\ \text{FLOPs}$, $\$0.25\ \text{USD}$, $8.5\ \text{Wh}$, and $5\ \text{minutes}$ of human prompt framing and reading versus $35.0\ \text{seconds}$ (up to several minutes for Deep Research) of autonomous thinking, representing 25% of users, 35% of token volume, and 30% of era spend[4].

Step 3 — Standardized Protocol Tool & Data Integration (MCP, L5-P3): Step Boundaries: Step 3 connects AI host applications (Claude Desktop, ChatGPT Desktop, Cursor, enterprise backends) to internal databases, CRMs, and APIs through standardized JSON-RPC 2.0 Model Context Protocol (MCP) servers exposing discoverable Tools, Resources, and Prompts (~25,000 tokens across multi-tool discovery and execution)[48], eliminating L_4's bespoke $N \times M$ vendor-locked glue code while remaining bounded to structured enterprise data transactions rather than open-ended multi-file code compilation (Step 4) or raw pixel screen clicking (Step 5)[4]. **Concrete Tool Stack:** Enterprise teams deploy the Model Context Protocol Python, TypeScript, Java, and C# SDKs (97M+ monthly downloads under the Linux Foundation AAIF), MCP Inspector, local stdio and remote streaming transports, Salesforce's Headless 360 MCP platform, Sourcegraph, and Gemini 3.5 Flash (scoring 83.6% on MCP Atlas multi-tool orchestration versus 69.5% for Sonnet 4.6)[49]. **Use Cases & Context Switching:** Adopted by 62% of production agent engineering teams in 2025 (*State of AI Engineering*) and exemplified by Salesforce's Headless 360 processing 4.5 million MCP calls in its first months live[48], Step 3 also requires enterprise architects to manage new protocol-level security challenges, such as the April 2026 OX Security disclosure of stdio remote code execution flaws across 200,000 unauthenticated MCP instances and tool-poisoning risks requiring strict OAuth and EU AI Act audit logging[48]. **Step Resource Use:** A single multi-tool workflow processes $\sim 25,000$ tokens, $\sim 1.5 \times 10^{15}\ \text{FLOPs}$, $\$0.12\ \text{USD}$, $4.5\ \text{Wh}$, and $4\ \text{minutes}$ of human verification versus $20.0\ \text{seconds}$ of runtime, capturing 15% of users, 20% of tokens, and

22% of era spend[4].

Step 4 — Autonomous Terminal & IDE Coding Agents (L5-P4): Step Boundaries: Step 4 grants an extended-thinking model direct autonomous control over a command-line terminal (CLI) or IDE agent harness equipped with six abstract primitives—bash, read, write, edit, glob, and grep—plus up to a 1,000,000-token context window and 64,000-token output ceiling[5], allowing it to autonomously explore a repository, write a reproduction script, edit multiple source files, run the test suite, and iterate across hundreds of turns (~250,000 tokens) until tests pass (SAE Level 4)[4]. **Concrete Tool Stack:** Developers run Anthropic Claude Code CLI, Cursor Composer 2, Mini-SWE-agent (Scale AI's standardized top harness), OpenAI GPT-5.2 Codex and GPT-5.4, Claude Sonnet 4.6 ($3/15 per 1M), Claude Opus 5.5 ($20/M output), Gemini 3.5 Flash (76.2% on Terminal-Bench 2.1), and high-value open or specialized coding models such as Qwen3.8-Flash-Next (62.5% SWE-bench Pro aggregate), GLM-5.2 (62.1%), and Poolside's Laguna S 2.1 (59.4% at $0.20/M output)[5]. **Use Cases & Context Switching:** Engineers delegate end-to-end GitHub issues, legacy framework migrations, and automated pull-request reviews (taught in DeepLearning.AI's Claude Code and Qodo courses)[11]; because unseen private commercial codebases reduce resolution accuracy by 10.0 to 22.5 percentage points compared to public open-source benchmarks, senior human engineers switch between delegating implementation loops to Step 4 Claude Code and conducting manual architectural verification[10]. **Step Resource Use:** A single autonomous coding ticket consumes $\sim 250,000$ tokens across 30–150 tool turns, $\sim 1.8 \times 10^{16}$ FLOPs, $\$1.10$ USD (with prompt caching and hybrid model routing), 45.0 Wh, and 8 minutes of human task specification and PR review versus 240.0 seconds (4 minutes) of autonomous CLI execution, representing 12% of users, 20% of token traffic, and 24% of era spend[4].

Step 5 — Full-System Multi-Agent Workflows & Computer Use (L5-P5): Step Boundaries: Step 5 is the October 2026 complexity ceiling—combining hierarchical multi-agent swarms (an orchestrator LLM dynamically spawning and synthesizing parallel worker subagents with isolated context buffers) and native graphical desktop and browser automation (Computer Use taking periodic screenshots, grounding pixel coordinates, moving the cursor, clicking, and typing across legacy software that lacks APIs or MCP servers)[7]—consuming $\sim 1,200,000+$ tokens and 15+ minutes per run, while remaining constrained by screenshot-inference-click latency ("using the computer in the dark for 3 to 5 seconds per click") and compounding cross-agent error costs[4]. **Concrete Tool Stack:** Teams orchestrate Anthropic's Computer Use API (Claude Sonnet 4.6 scoring 72.5% on OSWorld-Verified), Google Gemini 3.5 Flash (scoring 78.4% on OSWorld-Verified), OpenAI Operator / CUA (GPT-4o CUA and GPT-5.4), LangChain Deep Agents (launched July 2025 by Harrison Chase, hitting 9.9k GitHub stars in 5 hours during its March 2026 update), and adaptive trace-to-skill pipelines[18]. **Use Cases & Context Switching:** Deployed for overnight multi-agent project execution, cross-application enterprise ERP and claims processing across legacy Windows or web interfaces without APIs, and automated multimodal content pipelines[40]; while Gartner

forecasts 40% of enterprise applications will embed task-specific agents by the end of 2026, Gartner and MIT warn that over 40% of poorly scoped agentic projects risk cancellation by 2027 due to 800% cloud cost spikes when multi-agent loops collide in production[1]. **Step**

Resource Use: A single full-system run consumes $\sim 1,200,000$ tokens across orchestrator, worker subagents, and repeated high-resolution screenshot tokens, $\sim 9.0 \times 10^{16}\ \text{FLOPs}$, $\$6.50\ \text{USD}$, $220.0\ \text{Wh}$, and $20\ \text{minutes}$ of human orchestration and auditing versus $900.0\ \text{seconds}$ (15 minutes) of autonomous runtime, serving 3% of users while accounting for 5% of global token volume and 6% of era spend[4].
Full-Ladder Resource Totals (L5)

Aggregate Era Footprint (2024-09 to 2026-10): Across its 25-month window, L_5 reached $\sim 850\ \text{million}$ active users globally (with AI chip stock growing $3.4\times$ /year to pass $16\ \text{million}$ H100-equivalents and gigawatt-scale data centers costing $38 billion per GW), logging $\sim 65\ \text{billion}$ human hours[3]. Total era compute across frontier training (GPT-4.5, Claude 4/5, GPT-5, Gemini 3 reaching 10^{26}–$10^{27}\ \text{FLOPs}$) and massive test-time reasoning and agent inference reached $\sim 3.2 \times 10^{28}\ \text{FLOPs}$, consuming $\sim 68\ \text{million MWh}$ (68 TWh) of electricity, $\sim 380\ \text{trillion}$ tokens, and $\sim \$115\ \text{billion}$ in cumulative global enterprise ($37B in 2025 alone per Menlo Ventures) and consumer AI expenditure[2].

4. Part IV — Overall Patterns & Summary Visuals

Bottom-Line Summary: Examining all five historical ladders side by side exposes three universal laws governing AI evolution: the 15-month downward sinking cycle of middle-tier engineering steps into Step 1 default UI, the Jevons Paradox of inference token consumption where 100-fold price drops trigger 1,000-fold volume expansions, and the shift of human labor from low-level execution to high-level specification and verification[1]. **Verification Bottleneck:** Across every era, autonomous steps succeed in production only when paired with a fast, objective verification mechanism—such as automated unit tests in software coding (SWE-bench) or formal math checkers—explaining why coding captures 55% of departmental AI spend while unstructured organizational agents stall[2].

4.1. Long-Term Structural Patterns

Bottom-Line Summary: Four repeating structural dynamics connect L_1 through L_5: downward UI absorption, the Jevons Paradox of token volume, the split of general capabilities into specialized execution harnesses, and the human transition from typing to reviewing[2]. **Why**

Steps Sink: Whenever foundation labs increase context memory by $10\times$ or standardize an interface protocol, custom middleware written by application developers at Step 3 gets absorbed as a single button inside Step 1 consumer chat within 12 to 18 months[4]. **Why Spend Rises as Prices Fall:** Despite API token prices falling over 99% for equivalent capability between 2020 and 2026, enterprise AI spending grew 21.7-fold from $1.7B (2023) to $37.0B (2025) because test-time reasoning and autonomous agent loops multiplied single-task token consumption from 10^2 to 10^6+ tokens[2].

Pattern 1 — The 15-Month Step-Sinking Cycle: Advanced developer steps consistently migrate downward into the default consumer interface of the subsequent era[1]. In Era 1 (2018–2020), classifying sentiment or extracting entities required Step 4 GPU fine-tuning of BERT; in Era 2 (2020–2022), that became a Step 2 zero-shot davinci prompt; by Era 3 (2022–2023), it sank into Step 1 free browser chat[1]. Similarly, Era 3's signature engineering project—Step 3 custom Python + Pinecone PDF RAG—sank within 12 months into Era 4's Step 1 paperclip upload button (ChatGPT Plus and Claude 200k Projects), while Era 4's custom visualization scripts sank into Era 5's Step 1 Artifacts and MCP Apps[4]. Consequently, startups and in-house enterprise teams that build thin wrappers around current context limits face rapid getting-outdated risk unless they anchor on proprietary workflow state or vertical domain integration[2].

Pattern 2 — Jevons Paradox & Test-Time Compute: As Menlo Ventures explicitly observed in its 2025–2026 enterprise analysis, Jevons Paradox governs AI resource consumption: plunging per-token inference costs drive orders-of-magnitude expansions in total token demand[2]. When GPT-3 davinci cost $60.00 per million tokens in 2020, a $1,200-token few-shot call (0.072) was considered expensive, and a 250,000-token agent loop ($15.00) was commercially impossible[4]. Once hardware efficiency (+49% performance/dollar/year), model distillation (Flash and Haiku tiers), and prompt caching (50%–90% discounts) dropped effective input costs toward $0.18–0.75 per million tokens, developers unlocked Step 2 reasoning loops (35,000 tokens), Step 4 Claude Code repository sweeps (250,000 tokens), and Step 5 multi-agent GUI workflows (1,200,000 tokens), pushing single-task expenditure back up to $1.10–6.50 per complex job[3].

Pattern 3 — Verifiability Dictates Agent Autonomy: Comparing Epoch AI's compute trade-off laws with Anthropic's production engineering findings explains the uneven success of Step 4 and Step 5 agents across industries[7]. Epoch AI established that spending 5 to 6 extra orders of magnitude of inference compute (pass@k and iterative refinement) yields massive capability gains *only* in domains where candidate solutions can be verified cheaply and automatically—specifically software coding (via unit tests and compilers) and formal mathematics[7]. In coding, Claude Code or Cursor can run pytest via bash, inspect the exact traceback, and self-correct across 50 turns, propelling coding to $4.0 billion (55% of all departmental AI spend)[2]. Conversely, in ambiguous business workflows without automated verification tests, autonomous errors compound undetected, explaining why enterprises strongly prefer predefined Step 2 workflows or Step 3 MCP integrations and why 40%+ of

unconstrained non-coding agent pilots risk cancellation[1].

Pattern 4 — Shift of Human Effort from Execution to Specification & Audit: Tracking the ratio of human minutes to AI runtime seconds across the five ladders reveals a profound workplace reversal[1]. In Era 1 and Era 2, human practitioners spent 10 to 60 minutes preparing features or few-shot examples for an AI execution pass lasting 0.05 to 5.0 seconds[4]. By Era 5 Steps 4 and 5, the human spends 8 to 20 minutes upfront writing a crisp architectural specification (CLAUDE.md or design document) and reviewing the final pull request diff, while the AI executes autonomously for 240 to 900 seconds (4 to 15 minutes) across hundreds of file reads and test runs[4].

4.2. Cross-Era Comparison & Visual Summary

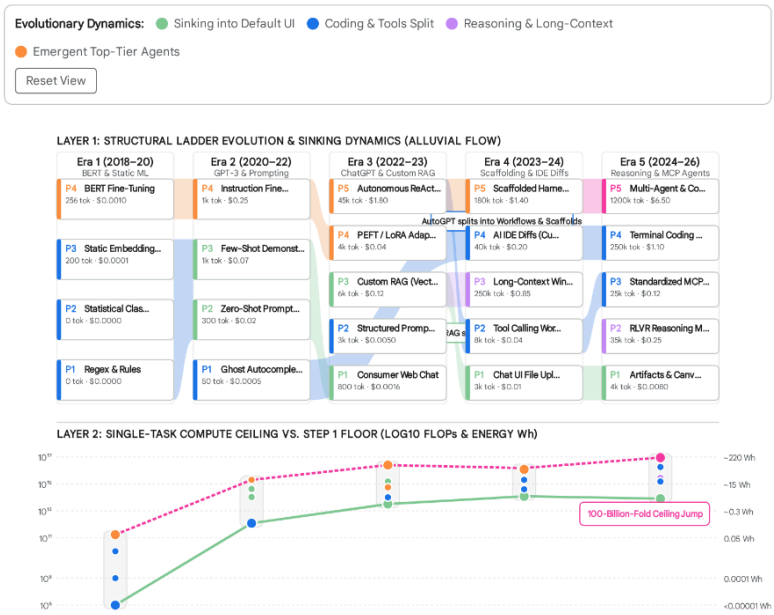
Bottom-Line Summary: The complete quantitative record across all 23 steps of Ladders L_1 through L_5 demonstrates that while Step 1 through Step 2 consistently host 70% to 85% of active users, the frontier of single-task compute has expanded 90-billion-fold from 10^6 FLOPs (L1-P1 regex) to 9.0×10^{16} FLOPs (L5-P5 multi-agent computer use)[4].

Master Synthesis Visual & Ledger: The compound visual and master ledger below connect every historical step across time, illustrating step sinking, splitting, single-task resource scaling, and era-wide macro totals from October 2018 to October 2026[2].

Eight Years of AI Ladders Pushed Yesterday's Engineering Stacks Into Default Chat While Multiplying Top-Step Compute 100-Billion-Fold

MASTER SYNTHESIS: STRUCTURAL TAXONOMY EVOLUTION, SINGLE-TASK COMPUTE, AND INDUSTRY SCALE

Cross-era step mapping from Era 1 (2018) to Era 5 (2026). Click any step to trace its historical lineage across eras.



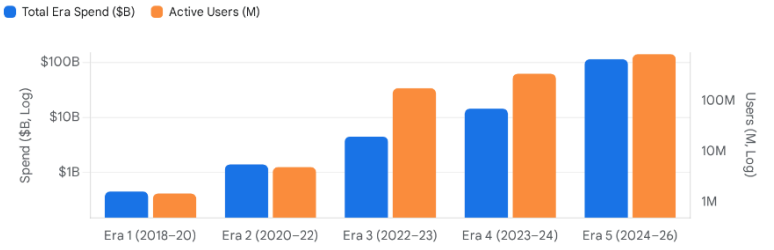
Era · Step P3 **Custom Retrieval-Augmented Generation (RAG)**

Click any step node in Layer 1 to lock evolutionary lineage

SINGLE-TASK COMPUTE	COST & LATENCY	ERA MACRO SHARE	LINEAGE TRAJECTORY
1.5 PFLOPs · 15.0 Wh 6,000 tokens per task	**$0.120 USD / task** 15 min human · 12.0s AI	**10% Users · 18% Tokens** 25% of Era AI Spending	Sank into Default Chat UI Era Total: $4.5B · 180M Users

Concrete Tool Stack & Real Use Case: Pinecone, FAISS, LangChain, LlamaIndex, sentence-transformers. Chunking, vector embedding, and similarity search over private PDF and wiki corpora.

LAYER 3: MACRO ERA TOTALS — INDUSTRY SPEND ($ BILLIONS) & ACTIVE USERS (MILLIONS)



Era 1	Era 2	Era 3	Era 4	Era 5
$0.45B · 1.5M	$1.4B · 5.0M	$4.5B · 180M	$14.5B · 350M	$115B · 850M
85k MWh	320k MWh	2.4M MWh	9.5M MWh	68M MWh

Across five structural eras from 2018 to 2026, advanced middle-tier engineering workflows repeatedly sank into Step 1 default consumer interfaces while new autonomous tiers emerged at the top. Single-task compute at the top of the ladder expanded from 170 billion FLOPs ($0.05 Wh) for a fine-tuned BERT call in Era 1 to 90 quadrillion FLOPs (220 Wh and 1.2 million tokens) for a multi-agent computer-use workflow in Era 5, driving cumulative era spending to $115 billion.

Data sources: First Movers (OpenAI AGI Levels), Anthropic Research, Menlo Ventures, State of AI, Epoch AI, SWE-bench Pro Leaderboard, Model Context Protocol

Era & Step ID	Step Name (Bottom-to-Top Rank)	Cross-Era Evolution Link (from Lt−1L_{t-1}Lt−1)	Single-Task Tokens	Single-Task Compute (FLOPs)	Single-Task Cost (USD)	Single-Task Energy (Wh)	Human Min / AI Sec	Era User / Token / Spend Share (%)
L_1 **-P1** (2018-10–2020-05)	Deterministic Rules & Regex	Baseline Software 1.0 Floor[4]	0	$1.0\times$	$\$0.000$	0.0000	$15\text{m}/0$	50% U [cite: 4]
L_1 **-P2** (2018-10–2020-05)	Feature-Engineered Classifiers	Classical ML (TF-IDF / XGBoost)[4]	0	$1.0\times$	$\$0.000$	0.0001	$30\text{m}/0$	30% U [cite: 4]
L_1 **-P3** (2018-10–2020-05)	Static Embeddings & BiLSTMs	Sequential Neural NLP (Word2Vec)[4]	200	$1.0\times$	$\$0.000$	0.002	$20\text{m}/0$	15% U [cite: 4]
L_1 **-P4** (2018-10–2020-05)	Fine-Tuned Transformer Encoders	**New Top Step** (BERT Transfer)[4]	256	$1.7\times$	$\$0.001$	0.05	$45\text{m}/0$	5% U/ [cite: 4]

L_2 **-P1** (2020-06–2022-11)	Inline Ghost-Text Autocomplete	**New Entry Step** (Codex / Copilot)[4]	50	$1.2\times$	$\$0.000$	0.02	$0.2\text{m}/0$	55% U [cite: 4]
L_2 **-P2** (2020-06–2022-11)	Zero-Shot Prompt Completion	**Combined** from L_1 P1–P3 (GPT-3)[4]	300	$1.05\times$	$\$0.018$	1.5	$3\text{m}/2.0$	25% U [cite: 4, 54]
L_2 **-P3** (2020-06–2022-11)	Few-Shot In-Context Chains	**Split/Expanded** from L_2 -P2 (CS324)[4]	$1,200$	$4.2\times$	$\$0.072$	6.0	$10\text{m}/5$	15% U [cite: 4, 54]
L_2 **-P4** (2020-06–2022-11)	Instruction & Head Fine-Tuning	**Evolved** from L_1 -P4 (T5 / davinci)[4]	$1,000$	$2.0\times$	$\$0.25$	30.0	$60\text{m}/2$	5% U/ [cite: 4]
L_3 **-P1** (2022-11–2023-11)	Consumer Web Chat Assistance	**Sank into Default UI** from L_2 P2/P3[1]	800	$3.2\times$	$\$0.001$	0.3	$2\text{m}/2.5$	65% U [cite: 4, 54]

L_3 **-P2** (2022-11–2023-11)	Structured Prompt Chaining	**Evolved** from L_2 -P3 (LangChain)[4]	$2,500$	$1.0\times$	$0.005	1.0	8m/6.0	20% U [cite: 4]
L_3 **-P3** (2022-11–2023-11)	Custom Vector-Database RAG	**New Step** (Pinecone / LlamaIndex)[4]	$6,000$	$1.5\times$	$0.12	15.0	15m/1	10% U [cite: 4]
L_3 **-P4** (2022-11–2023-11)	Parameter-Efficient Tuning (LoRA)	**Evolved** from L_2 -P4 (Llama-2 QLoRA)[4]	$4,000$	$5.6\times$	$0.04	8.0	45m/1	3% U/ [cite: 4]
L_3 **-P5** (2022-11–2023-11)	Unconstrained ReAct Agent Loops	**New Top Step** (Auto GPT / BabyAGI)[4]	$45,000$	$2.5\times$	$1.80	180.0	25m/1	2% U/ [cite: 4]
L_4 **-P1** (2023-11–2024-09)	Multimodal Chat & Built-In RAG	**Sank into Default UI** from L_3 -P3[4]	$3,000$	$1.2\times$	$0.015	0.5	3m/4.0	55% U [cite: 4]

L_4 **-P2** (2023-11–2024-09)	Deterministic Workflows & Tools	**Combined** from L_3 P2 & P5 (Workflows)[4]	$8,000$	$4.0\times$	$0.035	1.8	$5\text{m}/8.0$	22% U [cite: 4, 11]
L_4 **-P3** (2023-11–2024-09)	Long-Context In-Memory Analysis	**Split/Expanded** from L_3 -P3 (200k–2M)[4]	$250,00$	$1.5\times$	$0.85	35.0	$10\text{m}/3$	8% U/ [cite: 4, 55]
L_4 **-P4** (2023-11–2024-09)	Multi-File AI Code Editors & Diffs	**Split** from Coding (Cursor + Sonnet 3.5)[4]	$40,000$	$2.0\times$	$0.20	8.0	$4\text{m}/15$	12% U [cite: 4]
L_4 **-P5** (2023-11–2024-09)	Scaffolded Agent Harnesses	**Rebuilt** from L_3 -P5 (LangGraph/SWE)[4]	$180,00$	$1.2\times$	$1.40	50.0	$15\text{m}/1$	3% U/ [cite: 4, 13]
L_5 **-P1** (2024-09–2026-10)	Conversational Artifact & UI Apps	**Sank into Default UI** (Artifacts/M	$4,000$	$8.0\times$	$0.008	0.3	$3\text{m}/3.0$	45% U [cite: 4]

		CP UI)[4]						
L_5 **-P2** (2024-09–2026-10)	Test-Time Compute & Deep Research	**New Step** (RLVR o1/o3/ Opus thinking)[4]	$35,000$	$2.8\times$	$\$0.25$	8.5	$5\text{m}/35$	25% U [cite: 4]
L_5 **-P3** (2024-09–2026-10)	Standardized Protocol Tools (MCP)	**Re-Architected** from L_4 -P2 (AAIF MCP)[4]	$25,000$	$1.5\times$	$\$0.12$	4.5	$4\text{m}/20$	15% U [cite: 4]
L_5 **-P4** (2024-09–2026-10)	Autonomous Terminal Coding Agents	**Elevated** from L_4 P4/P5 (Claude Code)[4]	$250,00$	$1.8\times$	$\$1.10$	45.0	$8\text{m}/24$	12% U [cite: 4]
L_5 **-P5** (2024-09–2026-10)	Multi-Agent Swarms & GUI Use	**New Top Step** (OSWorld / Deep Agents)[4]	$1,200,$	$9.0\times$	$\$6.50$	220.0	$20\text{m}/9$	3% U/ [cite: 4]

Strategic Decision Rules: Engineering leaders and enterprise buyers allocating capital across the Era 5 ladder in 2026 can extract three operational directives from this historical trajectory[1].

Buy Commodity Steps: Because middle-tier capabilities sink into default consumer and SaaS

interfaces every 12 to 18 months—and because purchased or partner-built tools achieve a 67% production deployment rate versus 33% for custom in-house builds—organizations should purchase off-the-shelf seat licenses and connect internal data via standardized MCP servers rather than maintaining bespoke RAG infrastructure[2]. **Bound Unverified Workflows:** Deploy Step 2 predefined workflows and Step 3 MCP integrations for high-volume customer and back-office operations where unit-test verification does not exist and budgets are capped at $0.10–0.25 per run[4]. **Target Verifiable Autonomy:** Reserve high-compute Step 4 (Claude Code and Cursor) and Step 5 multi-agent budgets ($1.10–6.50 and 250k–1.2M tokens per task) for software engineering, automated quality assurance, and verifiable data science pipelines where automated test suites catch regressions and senior human reviewers audit private codebase diffs[4].

Works cited

1. The 5 AGI Levels: The Brutal Truth About Where We Are - First Movers, https://firstmovers.ai/agi-levels/
2. 2025: The State of Generative AI in the Enterprise - Menlo Ventures, https://menlovc.com/wp-content/uploads/2025/12/menlo_ventures_enterprise_ai_report-2025-123125.pdf
3. Trends in AI | Epoch AI, https://epoch.ai/trends
4. unknown_url
5. Agents and tools · Building with the Claude API, https://academy.claude.com/courses/building-with-the-claude-api/agents-and-tools
6. Claude Sonnet 4 vs Claude 3.5 Sonnet: Full Comparison, https://aicomparison.ai/claude-sonnet-4-vs-claude-3-5-sonnet/
7. Building Effective AI Agents - Anthropic, https://www.anthropic.com/engineering/building-effective-agents
8. Trading off compute in training and inference - Epoch AI, https://epoch.ai/publications/trading-off-compute-in-training-and-inference
9. Inference Scaling and AI Governance | Technical Report - GovAI, https://cdn.governance.ai/Inference_Scaling_and_AI_Governance_Technical_Report.pdf
10. SWE-bench Pro Leaderboard (September 2026) - Morph, https://www.morphllm.com/swe-bench-pro
11. Building Effective Agents: Practical Framework and Design Principles, https://www.zenml.io/llmops-database/building-effective-agents-practical-framework-and-design-principles
12. Generative AI at Work Statistics 2026 - Speakwise, https://speakwiseapp.com/blog/generative-ai-at-work-statistics
13. Raising the bar on SWE-bench Verified with Claude 3.5 Sonnet, https://www.anthropic.com/engineering/swe-bench-sonnet
14. Levels of AGI for Operationalizing Progress on the Path to AGI - arXiv, https://arxiv.org/pdf/2311.02462

15. What is an AI agent? - LangChain, https://www.langchain.com/blog/what-is-an-agent
16. 84% of developers use or plan to use AI tools, but more distrust their, https://www.reddit.com/r/developers/comments/1x3y1kz/84_of_developers_use_or_plan_to_use_ai_tools_but/
17. DeepLearning.AI Courses for AI Engineers: Which Free ... - Careery, https://careery.pro/blog/ai-careers/deeplearning-ai-courses-guide
18. Most of OpenAI's 2024 compute went to experiments - Epoch AI, https://epoch.ai/data-insights/openai-compute-spend
19. AI Energy Consumption Estimator, https://ninjalawyer.github.io/climateAI/
20. The 5 Levels of AI Autonomy Explained (2026 Framework) - AI Buzz, https://aibuzz.blog/the-5-levels-of-ai-autonomy/
21. Interpret and Apply the Gartner AI Maturity Model | Hupside, https://www.hupside.com/resources/how-to-interpret-and-apply-the-gartner-ai-maturity-model
22. Levels of AGI for Operationalizing Progress on the Path to AGI, https://www.alphaxiv.org/abs/2311.02462
23. I just read Anthropic's blog on 'Building effective agents', I love it. Is, https://www.reddit.com/r/ClaudeAI/comments/1hiww4y/i_just_read_anthropics_blog_on_building_effective/
24. Agentic Systems Explained: The Shift from Chatbots to Autonomous, https://www.raisesummit.com/post/agentic-systems-explained-shift-chatbots-autonomous-action
25. What's Software 3.0? (Spoiler: You're Already Using It) - Hugging Face, https://huggingface.co/blog/fdaudens/karpathy-software-3
26. Karpathy's Software 3.0: The Context Window Is Code, https://www.aibuilderclub.com/blog/karpathy-software-3-0
27. AGI (artificial general intelligence) - Sofokus, https://www.sofokus.com/ai-glossary/agi/
28. The 6 Levels of AI Autonomy (2026): A Practical Framework, https://www.agentys.io/en/blog/niveaux-autonomie-ia
29. What is "AGI" in the "AGI Era"? Clarifying a term used without a clear, https://note.com/obataka123/n/nb19a39c32951?hl=en
30. From Narrow AI to AGI: Types of AI, Technical Architectures, and, https://devgenxai.com/blog/types-of-ai-roadmap-to-agi-artificial-general-intelligence-2026/
31. Levels of AGI: Operationalizing Progress on the Path to AGI, https://montrealethics.ai/levels-of-agi-operationalizing-progress-on-the-path-to-agi/
32. Levels of AGI for Operationalizing Progress on the Path to AGI - arXiv, https://arxiv.org/abs/2311.02462
33. how-to-build-effective-ai-agents-anthropic.pdf - HKDCA, https://www.hkdca.com/wp-content/uploads/2025/02/how-to-build-effective-ai-agents-anthropic.pdf
34. building-effective-agents.md - GitHub,

https://github.com/machinedge/building-effective-agents/blob/main/building-effective-agents.md
35. Building Effective Agents by Anthropic | PDF | Workflow - Scribd, https://www.scribd.com/document/811579416/Building-Effective-Agents-by-Anthropic
36. The 5 Levels of AI Agent Autonomy Explained, https://pickaxe.co/post/levels-of-ai-agent-autonomy
37. A Survey of Autonomous Research Agents - Deli Chen, https://victorchen96.github.io/auto_research/auto_research_survey.pdf
38. What is a "cognitive architecture"? - LangChain, https://www.langchain.com/blog/what-is-a-cognitive-architecture
39. LangChain Deep Agents: Build Autonomous AI Agents to Automate, https://emelia.io/hub/deep-agents-langchain-autonomous-ai-agents
40. The State of AI, https://www.stateofai.app/
41. OpenAI Comprehensive Report | PDF | Intelligence (AI) & Semantics, https://www.scribd.com/document/1065994523/OpenAI-Comprehensive-Report
42. Build, buy or partner: how to choose your path to AI - Xcope, https://xcope.ai/insights/build-buy-or-partner/
43. Generative AI Development Company | Foreignerds, https://foreignerds.com/generative-ai-development/
44. AI Courses - DeepLearning.AI, https://www.deeplearning.ai/courses
45. DeepLearning.AI Short Courses: 2026 Review - TinkerLLM, https://tinkerllm.com/blog/deeplearning-ai-short-courses-review/
46. CS324 - Large Language Models, https://stanford-cs324.github.io/winter2022/
47. Reports | Menlo Ventures, https://menlovc.com/reports/
48. MCP Model Context Protocol for Business | 2026 Guide - NAXIA, https://www.naxiaglobal.com/blog/mcp-model-context-protocol-business-en/
49. Claude Sonnet 4.6 vs Gemini 3.5 Flash: Best Value Coding Model?, https://codingfleet.com/blog/claude-sonnet-4-6-vs-gemini-3-5-flash/
50. Learn local AI — Will-It-Run, VRAM, quantization, and best models, https://www.runlocalai.co/learn
51. CS 6501 Workshop on Building AI Agents, https://www.cs.virginia.edu/~rmw7my/Courses/AgenticAISpring2026/agenticAI.html
52. Agentic AI short courses - DeepLearning.AI, https://www.deeplearning.ai/courses?types=short_course
53. Inference Scaling Reshapes AI Governance - arXiv, https://arxiv.org/pdf/2503.05705
54. OpenAI API Pricing (Updated 2026) – All Models & Token Costs, https://pricepertoken.com/pricing-page/provider/openai
55. Prompt Caching Infrastructure: Reducing LLM Costs and Latency, https://introl.com/blog/prompt-caching-infrastructure-llm-cost-latency-reduction-guide-2025
56. LLM Caching Strategies: Prompt Caching, Semantic ... - NeuralTrust AI, https://neuraltrust.ai/blog/llm-caching-strategies

57. Claude 3.5 Sonnet Benchmarks & Pricing (October 2026), https://benchlm.ai/models/claude-3-5-sonnet
58. Model Context Protocol - Wikipedia, https://en.wikipedia.org/wiki/Model_Context_Protocol
59. Claude 3.5 Sonnet updated, and Claude 3.5 Haiku soon - Discussions, https://forum.cursor.com/t/claude-3-5-sonnet-updated-and-claude-3-5-haiku-soon/24165
60. LangChain's Harrison Chase on Building the Orchestration Layer for, https://sequoiacap.com/podcast/training-data-harrison-chase
61. AI Courses - DeepLearning.AI, https://www.deeplearning.ai/courses?types=course
62. Partnering with LangChain: The LLM Application Framework, https://sequoiacap.com/article/partnering-with-langchain-the-llm-application-framework
63. What is Model Context Protocol (MCP)? The Universal Standard for, https://resources.rework.com/libraries/ai-terms/model-context-protocol
64. What is MCP? The Model Context Protocol Explained - DataWalk, https://datawalk.com/what-is-mcp-the-model-context-protocol-explained/
65. Trends in the dollar training cost of machine learning systems, https://epoch.ai/publications/trends-in-the-dollar-training-cost-of-machine-learning-systems
66. DeepMind's 6 Levels of AGI: From Calculator to Superhuman, https://www.youtube.com/watch?v=tEXHO-kXgWE
67. OpenAI vs Anthropic Prompt Caching: Key Differences - Ginger Labs, https://gingerlabs.ai/blog/openai-vs-anthropic-prompt-caching
68. SWE-bench Verified Leaderboard and Methodology - Vals AI, https://www.vals.ai/benchmarks/swebench
69. Engineering AI systems with Model Context Protocol · Raygun Blog, https://raygun.com/blog/announcing-mcp/
70. Introducing the Model Context Protocol - Anthropic, https://www.anthropic.com/news/model-context-protocol
71. Claude 3.5 Sonnet (2024-10-22) — 2024 AI Agent Index, https://aiagentindex.mit.edu/2024/claude-3-5-sonnet-2024-10-22/
72. What is Model Context Protocol (MCP)? A guide | Google Cloud, https://cloud.google.com/discover/what-is-model-context-protocol
73. Managing Prompts as Enterprise Assets: A Portfolio Approach, https://medium.com/@segev_shmueli/managing-prompts-as-enterprise-assets-a-portfolio-approach-e9552e24f327